Knowledge-Based Systems 250 (2022) 109075

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Multi-objective particle swarm optimization with multi-mode collaboration based on reinforcement learning for path planning of unmanned air vehicles

Xiangyin Zhang [a,b,*], Shuang Xia [a,b], Xiuzhi Li [a,c], Tian Zhang [a,b]

[a] Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China
[b] Beijing Institute of Artificial Intelligence, Beijing 100124, China
[c] Engineering Research Centre of Digital Community, Ministry of Education, Beijing 100124, China

ARTICLE INFO

Article history:
Received 25 August 2020
Received in revised form 14 May 2022
Accepted 16 May 2022
Available online 23 May 2022

Keywords:
Path planning
Multi-objective particle swarm optimization
Unmanned air vehicle
Reinforcement learning

ABSTRACT

In order to solve the multiple unmanned aerial vehicles (UAVs) collaborative path planning problem under complex environments with multiple constraints, the multi-objective particle swarm optimization algorithm with multi-mode collaboration based on reinforcement learning (MCMOPSO-RL) is proposed in this paper to find optimal paths and handle constraints simultaneously. Reinforcement learning (RL) is applied to enable the proposed algorithm to choose the suitable position updated mode to achieve the high performance. Multi-mode collaboration strategy is developed to update the particle positions, where three modes are designed to balance the population diversity and the convergence speed, including the exploration, exploitation modes, and the hybrid update mode. Experimental results show that MCMOPSO-RL can solve the path planning problem for multiple UAVs more efficiently and robustly than other algorithms.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Unmanned aerial vehicle (UAV) is an aircraft that does not require human driving onboard, which can effectively complete boring and dangerous tasks. UAVs have been widely used in many fields [1], including target tracking [2], land mapping [3], search and rescue [4], et al. With the changes of the times and the advancement of technology, the tasks that need to be completed by UAVs are becoming more and more complex. In practical applications, due to high energy consumption, scattered locations, and numerous operating equipment, a single UAV may not be able to successfully complete all tasks. The multiple UAVs cooperation becomes a current research hotspot.

The mission planning of multiple UAVs needs to consider many factors, including the allocation between the UAVs and the targets, the planning of the flight paths, the coordination of arrival time, and so on. Path planning of multiple UAVs is one of the challenging problems, involving high dimensionality, equality and inequality constraints, and the requirements of spatial–temporal cooperation of multiple UAVs. The goal of path planning is to make UAVs fly to targets distributed in different positions with the shortest total flight distance. In the flight environment, there are some threat areas that UAVs cannot enter. Meanwhile, collisions between UAVs need to be avoided. In addition, the time for the UAVs to reach the targets needs to be coordinated in many cases. For example, in a combat mission, all UAVs need to reach the corresponding target points at the same designated time, or in a transportation mission, multiple UAVs arrive at the same target point at the same time interval. Many traditional methods, such as A* algorithm [5], artificial potential field [6] and random trees [7] are used to solve the problem. But these methods usually suffer from high time complexity, when UAV path planning has multiple constraints. Therefore, meta-heuristic algorithms, including particle swarm optimization (PSO) [8], genetic algorithm (GA) [9], ant colony optimization (ACO) [10], artificial bee colony algorithm (ABC) [11], fruit fly optimization algorithm (FOA) [12], differential evolution (DE) [13], pigeon-inspired optimization (PIO) [14], are gradually applied to solve the path planning problem.

In recent years, the research on the path planning of multiple UAVs has been intensified. Researchers have developed various path planners based on intelligent optimization algorithms. Li et al. [15] proposed a method for path planning of multiple UAVs with online changing tasks. It uses the reference point mechanism and a judgment matrix to improve the computational accuracy and stability of the traditional fruit fly optimization algorithm. Meanwhile, the task change is taken into consideration and the task assignment with task priority is realized. Liu et al. [16]

* Corresponding author at: Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China.
E-mail address: zhangxgyn@foxmail.com (X. Zhang).

https://doi.org/10.1016/j.knosys.2022.109075
0950-7051/© 2022 Elsevier B.V. All rights reserved.

proposed a four-dimensional coordinated collision free path planning method for multiple UAVs. It designs the spatial refined voting mechanism for the standard particle swarm algorithm, which overcomes the shortcomings of local optimization and slow convergence. Furthermore, this method takes time variables into consideration, and successfully generates collision free paths for multiple UAVs with the same time consumption. Radmanesh et al. [17] used the gray wolf algorithm to plan the flight paths of multiple UAVs. It builds a risk map by adopting the distance based value function and Bayesian update method, which effectively solves the problem of dynamic obstacle avoidance in an uncertain environment.

In this paper, the UAV path planning problem is regarded as a constraint problem. In order to solve this problem more effectively, the constraint handling method is crucial. The penalty function [18] is the most commonly used method, which transforms the constrained problem into an unconstrained problem by employ constraint violations as penalty terms. The problem with the penalty function method is that it is difficult to determine the appropriate penalty factor. The feasibility rule [19] treats the feasible solutions and the infeasible solutions separately, and the feasible solutions are superior to the infeasible solutions. However, when the optimal solutions locate at the boundary of the feasible region, this method may not find them. Besides, the multi-objective optimization techniques [20,21] can also handle the constraint problem well, which regards constraint conditions as objective functions, and thereby, transforming the constrained problem into a multi-objective unconstrained optimization problem.

In recent years, multi-objective optimization algorithms have achieved good results in trajectory planning. An evolutionary optimal control solver that applies a multiple-shooting discretization technique with the NSGA-III optimization algorithm is constructed in [22]. To solve the multi-objective trajectory optimization problem, an adaptive differential evolution algorithm combined with a modified game theory method was proposed in [23]. It not only accelerates the convergence speed, but also embeds the control logic to improve the quality of the generated solutions. Among the multi-objective algorithms, multi-objective particle swarm optimization (MOPSO) has been applied in many fields [24,25] due to its simplicity and fast convergence [26], for example, path planning problem [27]. S.Z. Zhao, et al. [28] proposed the two-lbests based MOPSO algorithm. By dividing each objective function range in the external archive into several bins, the search is concentrated in a small area in the parameter space in the vicinity of the best existing fronts. K.M. Ang, et al. [29] proposed the constrained multi-swarm PSO without velocity, which combines the two diversity maintenance schemes, multi-swarm technique and probabilistic mutation operator, to prevent premature convergence of the algorithm. Z.K. Feng, et al. [30] applied quantum behavior into MOPSO, which uses the elite archive to conserve the Pareto optimal solutions, and meanwhile, the neighborhood searching and chaotic mutation strategies are used to enhance the search capability and diversity of population. These improved MOPSO variants show good performance on many optimization problems. However, the particles in the population are updated in the same way, without taking into account the characteristics of each particle itself in the search process.

Reinforcement learning (RL) is an important research orientation in the field of artificial intelligence [31]. RL rewards or penalizes the system through performance evaluation, so as to obtain optimal control decisions under different environmental conditions. B. Li, et al. [32] uses RL to guide the behavior of UAV to track the target through the setting of reward and punishment methods. X. Zhou, et al. [33] design a path planner based on deep reinforcement learning, which is successfully applied to the path planning of unmanned surface vehicles. To realize the path planning of the UAV in three-dimensional space, C.Z. Qu, et al. [34] combined RL and the gray wolf optimization algorithm, and experimental results show that RL has strong practicability in solving path planning problems. Based on the multi-objective algorithm, R. Chai, et al. [35,36] used a deep neural network to feed back the optimal control action in the real-time system, which realized the trajectory planning of hypersonic vehicle reentry flight. The difference in this paper is that we use reinforcement learning to improve the algorithm's ability to search for optimal solutions.

In this paper, a multi-objective particle swarm optimization algorithm with multi-mode collaboration based on reinforcement learning (MCMOPSO-RL) is proposed to solve the UAV path planning problem. First, the constraint problem is transformed into a multi-objective optimization problem, and then MCMOPSO-RL is adopted to plan the UAV flight path offline. The proposed algorithm organically integrates MOPSO and RL. The particles in MOPSO are regarded as agents in RL, and the update modes of particles are chosen by the cumulative performance. For each particle, three update modes are developed: exploration, exploitation and hybrid mode. In exploration and exploitation update mode, a speed update operator with Gaussian distribution is applied to improve the update strategy of particles. Moreover, global search and local search are performed respectively to improve population diversity and accelerate convergence. The hybrid mode combines quantum behavior and mutation strategy as a routine search operation. The optimal path of the UAV in a complex three-dimensional environment can be found based on the proposed algorithm. Experimental results show that MCMOPSO-RL can more effectively solve the UAV path planning problem than other algorithms.

The rest of this paper is organized as follows. Section 2 is the related work, briefly introducing multi-objective optimization problem, quantum-behaved particle swarm optimization and reinforcement learning. In Section 3, the MCMOPSO-RL is described in detail. Section 4 describes the path planner based on MCMOPSO-RL, including path representation, mathematical model and the specific implementation process of MCMOPSO-RL based path planning method. In Section 5, experiments and comparisons with some other algorithms are provided. Finally, Section 6 concludes this paper.

2. Related work

2.1. Multi-objective optimization problem

The general multi-objective optimization problem usually contains several conflicting objective functions, and the result is a set of compromise solutions, namely Pareto optimal solution set, whose corresponding set in the function space is called Pareto front (PF). The multi-objective optimization problem can be expressed mathematically as:

$$\min \boldsymbol{f}(x) = [f_1(x), f_2(x), \ldots, f_k(x)] \tag{1}$$

where $f_i(x)$ $(i = 1,2, \ldots, k)$ is the objective function, k is the number of objective functions.

In the search process, the multi-objective algorithm will produce a set of solutions. Unlike single-objective optimization problem, it has multiple objective function values, and cannot be indicated which solution is better by simply comparing an objective function value. Therefore, the concept of dominance is introduced. When solution x_1 is not worse than x_2 in all objectives, it is called x_1 dominates x_2 ($x_1 \prec x_2$ $\prec$ x_2). Once the dominant solution is found, the Pareto optimality is applied to get a non-dominant set of solutions. For the multi-objective particle swarm algorithm, these non-dominated solutions are stored as the positions of the best particles.

2.2. Quantum-behaved particle swarm optimization

Birds flock foraging is achieved through collective collaboration between birds. PSO searches for potential solutions in a multidimensional space by simulating this process.

PSO begins by initializing the velocities and positions of particles and then iterates to find the optimal solution. In each iteration, the particle updates its velocity and position by tracking two "extreme values" (the personal best position x_{pbest} and the global best position x_{gbest}). After finding these two optimal values, the velocity v_i and position x_i of the ith particle are updated as follows:

$$v_i(t+1) = \omega \cdot v_i + c_1 \cdot r_1 \cdot (x_{\text{pbest},i}(t) - x_i(t)) + c_2 \cdot r_2 \cdot (x_{\text{gbest}}(t) - x_i(t)) \quad (2)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (3)$$

where t is iteration count; ω is inertia weight; c_1 and c_2 are learning factors; r_1 and r_2 are uniformly generated random numbers in the range of [0,1].

In order to solve the problem that the classic PSO algorithm is easy to fall into local optimization, Sun et al. [37] introduced the concept of quantum mechanics and proposed the quantum-behaved particle swarm optimization algorithm (QPSO) to improve the global optimization ability. In quantum space, each particle has quantum behavior, and the velocity and position of the particles cannot be determined simultaneously. Thus the state of a particle is no longer described by its position and velocity, instead it is the wave function $\psi(x, t)$ of the particle. The probability density of particles appearing at a certain point x in space can be solved by Schrödinger equation, and then the Monte Carlo method can be used to obtain the particle motion iteration equation as follows:

$$\begin{cases} x_i(t+1) = p_i + \beta \cdot |mbest - x_i(t)| \cdot \ln\left(\frac{1}{u}\right) & \text{if } \theta \geq 0.5 \\ x_i(t+1) = p_i - \beta \cdot |mbest - x_i(t)| \cdot \ln\left(\frac{1}{u}\right) & \text{if } \theta < 0.5 \end{cases} \quad (4)$$

where p_i is the local attractor; β is the contraction–expansion coefficient; *mbest* is a global point called mean best, which defined as the mean of the personal best positions of all particles; u and θ are uniformly generated random numbers in the range of [0,1].

The average best location *mbest* is given by:

$$mbest = \frac{1}{M}\sum_{i=1}^{M} x_{pbest,i} \quad (5)$$

where M represents the population size of particles.

The parameter β is designed to control convergence rate. The most commonly used method to adjust β is a linear change:

$$\beta = \beta_{max} - (\beta_{max} - \beta_{min}) \cdot \frac{t}{t_{max}} \quad (6)$$

where t_{max} is the maximum number of iterations, β_{max} and β_{min} are the maximum and minimum values of parameter β, respectively.

The local attractor p_i is calculated as follows:

$$p_i = \frac{\varphi_1 \cdot x_{pbest,i} + \varphi_2 \cdot x_{gbest}}{\varphi_1 + \varphi_2} \quad (7)$$

where φ_1, φ_2 are uniformly generated random numbers in the range of [0,1].

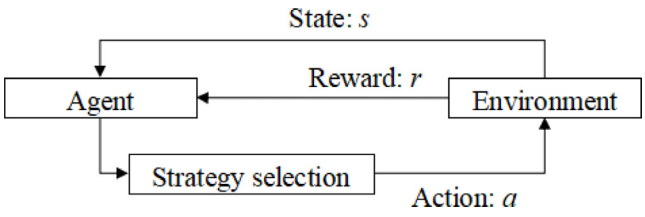


Fig. 1. The structure of Q-Learning algorithm.

2.3. Reinforcement learning

Reinforcement learning is an important method of machine learning and has a wide range of applications in the field of autonomous decisions. It can greatly reduce the interference of researchers to the model. The agent interacts with the environment continuously, and learns the optimal decision strategy through trial and error. Q-Learning is one of the most commonly used algorithms in reinforcement learning. It adopts a time difference method to update the current value function according to the expected reward of the next state.

Q-Learning involves an agent, Q-table, state set **S**, and action set **A**. The agent perceives the state s ($s \in \mathbf{S}$) from the environment and makes an optimal action strategy ($a \in \mathbf{A}$) according to the Q-table. Applying this action to the environment, the state s will be changed, and the reward value r will be fed back to the agent from the environment, and then the Q-table will be updated. Iterate in this way until the agent gets the maximum reward value. The structure of Q-Learning is shown in Fig. 1.

The Q-table is updated based on the reward dynamically as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \lambda\left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)\right] \quad (8)$$

where $Q(s_t, a_t)$ is the Q-value when the action a_t is performed at the state s_t; λ is the learning rate; γ stands for the discount factor; r_{t+1} is the immediate reward obtained from the performance of a_t at the state s_t.

3. The proposed algorithm

3.1. The structure of MCMOPSO-RL

In order to solve complex optimization problems, all metaheuristic algorithms need to balance between global search ability and local search ability. However, sometimes a unified search process does not guarantee a global optimal solution. Independent search operations will have better performance, and reinforcement learning can easily achieve this goal.

In MCMOPSO-RL, the particles are regarded as the agents of reinforcement learning. The entire search space is the interactive environment, the state s is the position update mode of the previous iteration of the particle, and the change of state s is regarded as the action a. The framework of MCMOPSO-RL algorithm is shown in Fig. 2.

Each particle has three operations to update its position: exploration, exploitation and hybrid update mode. According to the accumulated performance, each particle selects its position update mode adaptively. If the updated particle performs better, a positive reward will be given, otherwise the negative reward is given to complete the punishment.

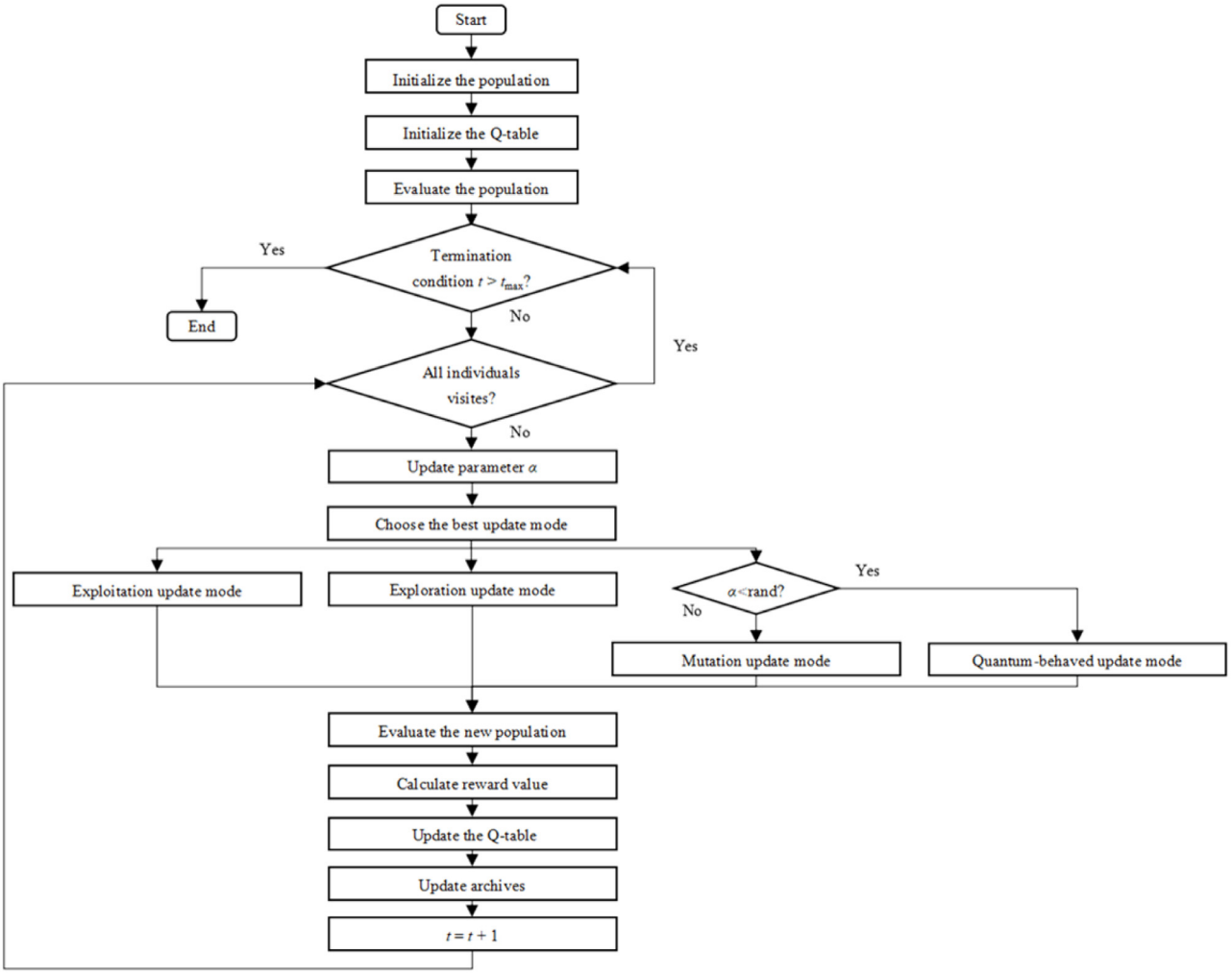


Fig. 2. The framework of MCMOPSO-RL algorithm.

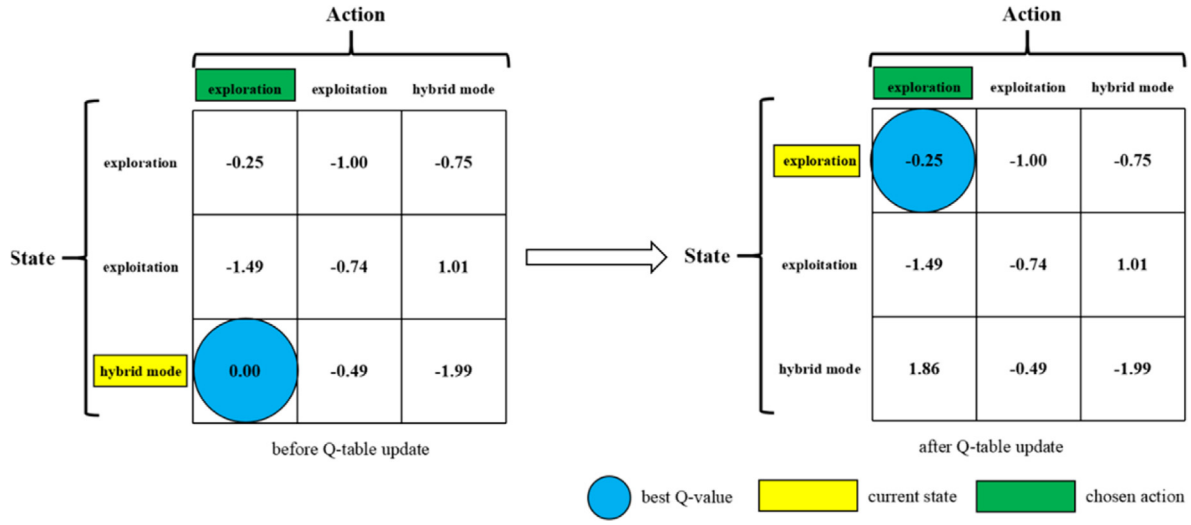


Fig. 3. The update process of the Q-table.

In the proposed algorithm, the Q-table is designed as a 3×3 matrix. Its columns represent the action, and rows represent the state. Each individual has its own Q-table to ensure that the learning process is independent. In Fig. 3, a particle is currently in a "hybrid mode" state, and its "exploration" action will get the greatest expected reward. After the action is taken, the reward

can be calculated according to the new position of the particle, and then the Q-value in the table will be updated by Eq. (8). Finally, this particle enters a new state "exploration".

In Q-Learning algorithm, the setting of the learning rate λ is very important. The greater the learning rate, the higher the weight of future rewards; the smaller the learning rate, the immediate rewards account for the main part. Therefore, the learning rate can adaptively be reduced during the iterative process, which can effectively learn from the search space.

In this paper, the learning rate is updated as follows:

$$\lambda = \frac{\lambda_{initial} + \lambda_{final}}{2} - \frac{\lambda_{initial} - \lambda_{final}}{2} \cdot \cos\left(\pi\left(1 - \frac{t}{t_{max}}\right)\right) \tag{9}$$

where $\lambda_{initial}$, λ_{final} are represent the initial and final value of λ respectively, t is defined as the current iteration number, t_{max} is defined as the maximum iteration number.

The reward r is set as follows:

$$r = \begin{cases} 1 & if\ the\ fitness\ improved \\ -1 & otherwise \end{cases} \tag{10}$$

3.2. The exploration and exploitation update mode

The contradiction between global search ability and local search ability restricts the performance of the algorithm. In MCMOPSO-RL, the exploration mode performs a global search to find the possible range of the optimal solution. In contrast, the exploitation mode performs a local search to find the optimal solution within the possible range.

Eq. (2) is the velocity update method of particles applied to the single-objective optimization problem. In the formula, x_{gbest} is the global optimal solution. However, it is replaced by a set of solutions, saved in the external archives, in the multi-objective optimization problem. Meanwhile, sometimes replacing uniformly distributed random numbers with Gaussian random numbers works well [38]. Therefore, a speed update operator with Gaussian distribution is adopted to modify Eq. (2). Then the velocity update method of the particles in the exploration and exploitation update mode is obtained as follows:

(1) *Exploration update mode*

$$v_i(t+1) = \omega \cdot v_i + c_{11} \cdot r_1 \cdot (x_{\text{pbest},i}(t) - x_i(t)) + c_{21} \cdot g_1 \cdot (x_b(t) - x_i(t)) \tag{11}$$

where x_b is a random individual in the external archives, and g_1 is generated with Gaussian distribution. The learning factor c_{11} should be large and c_{21} should be small, so that the historical experience of the particles themselves can more mainly guide the search direction and enhance the diversity of the population.

(2) *Exploitation update mode*

$$v_i(t+1) = \omega \cdot v_i + c_{12} \cdot r_1 \cdot (x_{\text{pbest},i}(t) - x_i(t)) + c_{22} \cdot g_1 \cdot (x_b(t) - x_i(t)) \tag{12}$$

where the learning factor c_{12} should be small and c_{22} should be large, so that the optimal particles can guide the search direction more, and the algorithm can converge to the optimal solution more quickly.

3.3. The hybrid update mode

Both exploration and exploitation modes focus on the ability of a certain aspect of the algorithm. As a supplement, a hybrid update mode is developed as a routine operation.

(1) *Mutation update mode*

In the hybrid update mode, there is more possibility at the beginning of the iteration to use mutation operation to increase population diversity as follows:

$$x(t+1) = x(t) + F \cdot (x_b - x_r) \tag{13}$$

where x_b is a random individual in the external archives, x_r is a random individual in the current particles, F is the mutation factor.

(2) *Quantum-behaved update mode*

In the later stage of the iteration, the quantum-behaved update method is more likely to have a good performance. It not only has strong local search ability, but also has a certain randomness of the particle position, which ensures the diversity of the population. So, the update method of the position in the quantum particle swarm is modified as follows to make it suitable for the multi-objective optimization problem.

$$p_i = \varphi \cdot x_{pbest,i} + (1 - \phi) \cdot x_b \tag{14}$$

where x_b is a random individual in the external archives, φ is a random number between 0 and 1.

(3) *Mode conversion factor*

The mode conversion factor α is applied to decide which update mode is adopted. When α is less than a random number between 0 and 1, quantum-behaved update mode is adopted. Otherwise, the mutation update mode is adopted. The mode conversion factor α is calculated as follows:

$$\alpha = \frac{1}{1 + exp(4 - \varepsilon \cdot t / t_{max})} \tag{15}$$

where $exp()$ is an exponential function with e as the base, $\varepsilon = 20$ is a parameter to control the increasing rate of α, t is iteration count, $t_{\max}$ is the maximum number of iterations.

In this way, at the beginning of the iteration, the value of α is small, and there is a greater probability of adopting the mutation update mode. At the later stage of the iteration, the value of α is closer to 1, and the quantum-behaved update mode is more inclined.

3.4. Updating archives

The external archives are used to retain the non-dominated solutions generated during the search process. After the iteration is over, it is the optimal solution set obtained by the optimization algorithm. The update strategy for external archives is shown in Fig. 4.

3.5. Computing complexity

Suppose that N is the number of individuals involved in the calculation, P is the number of objective functions, M is the maximum capacity of external archives, $t_{\max}$ is the maximum number of iterations. The MCMOPSO-RL algorithm can be simply divided into two phases. The first phase is the initialization phase, which is only executed once at the beginning of the algorithm and its computing complexity is $O(N)$. The second phase is the optimization phase, which is implemented during each iteration and can be divided into three parts: individual update, Q-table update and archives update. The computing complexity of each part is shown as follows:

(1) *Individual update*

In the individual update part, each individual can independently choose one of the three update modes based on its accumulated performance. In order to calculate the extreme computing complexity of each iteration, it is assumed that all individuals perform the same update mode. For exploration and exploitation modes, $O(N)$ computations are required to update the position

Steps of updating external archives
*/*Input parameters*/*
The current capacity of external archives m, the maximum capacity of external archives M
*/*Update Archives*/*
For each particle in current population x_i **do**
  For each particle in current population x_j **do**
    If $x_i \prec x_j$ **do**
      Delete x_j from the archives
    End if
  End for
  If x_i is not dominated by any solution in the archives **do**
    Place x_i in the archives.
  End if
End for
If $m > M$ **do**
  Calculate the crowding distance of all solutions in the archives and arrange them in descending order.
  Delete the solutions with the smaller crowding distance until $m=M$.
End if

Fig. 4. The update strategy for external archives.

of each particle according to Eqs. (11) and (12), respectively. When all particles adopt the hybrid update mode, one of the quantum-behaved update method and the mutation method is chosen. While, for both of them, $O(N)$ computations are required. In general, the computing complexity of the individual update part is $O(N)$.

(2) *Q-table update*

The fitness value of each particle after position update is compared with the previous value to calculate the learning reward value and its computing complexity is $O(N)$. Since each particle corresponds to a Q-table, the computing complexity of Q-table update is also $O(N)$. Therefore, the computing complexity of the Q-table update process is $O(N)$.

(3) *Archives update*

In the archives update part, non-dominated sorting is the main factor affecting computing complexity. To calculate the extreme complexity, it is assumed that the particles in the whole update process are non-dominated solutions and the archives is full. First, each particle in the population must be compared with other particles at each objective function, and its computing complexity is $O(PN^2)$. Then, each particle in the population is compared with all particles in the archives, and the computing complexity is $O(PNM)$. Finally, the crowding distance is calculated, and its computing complexity is $O(P(M+N)\log(M+N))$. Therefore, the computing complexity of the archives update process is $O(P(M+N)(N+\log(M+N))$.

In summary, the maximum computing complexity of each iteration of MCMOPSO-RL is $O(P(M+N)(N+\log(M+N))$. The total computing complexity is $O(t_{max}\cdot P(M+N)(N+\log(M+N))$, which is the same as the traditional MOPSO algorithm. It proves that our improvement to the algorithm does not increase the computing complexity and still maintains a high execution speed.

4. Path planning based on MCMOPSO-RL

4.1. Path representation

As a key component of UAV mission planning, UAV path planning provides the most powerful technical guarantee for autonomous control and intelligent flight of UAV. Its goal is to plan a flight path with a higher safety and a lower trajectory cost from the starting point to the target point based on UAV's own maneuverability and flight space environment. In addition, for multiple UAVs, collision avoidance between them should be considered.

In this paper, it is assumed that the flight environment is fixed, and all threat areas and forbidden flying areas are known. In the three-dimensional flight space, the UAV path is described in order of the flight starting point S, N waypoints and target point T as $Path = \{S, P_1, P_2, \ldots, P_N, T\}$, which can be determined by n control points w_i: (x_i, y_i, z_i) and a predefined trajectory smooth strategy, as shown in Fig. 5. This has an important quantitative role in evaluating the planned paths for UAVs. The control points can be any point in space, which allows UAVs to bypass the threat areas by moving backwards. In order to ensure the flight ability and smoothness of the UAV flight paths, the Bezier curve is usually adopted. It can describe better paths with fewer parameters. In this paper, the cubic B-spline curve is applied to describe the flight paths of UAVs.

4.2. Mathematical model of UAV path planning

The performance indicators for evaluating flight paths of UAV mainly include flight cost and the safety of flight. The flight cost is based on the length and altitude of the flight. The safety of flight is the constraint to be satisfied in the optimization problem. Flight safety should be guaranteed as much as possible in the path planning, which mainly considers the threat of the dangerous area, forbidden flying area, terrain, the turning angle, climbing & gliding slope constraint and the risk of collision with other UAVs.

Assume that the flight path is represented by a collection of discrete waypoints $\{p_0, p_1, p_2, \ldots, p_N, p_{N+1}\}$ and the coordinate of p_k is (x_k, y_k, z_k), where the first waypoint is the start point and the last waypoint is the target point. Therefore, the performance index of the UAV path is calculated as follows:

(1) *Length cost*

When the flight speed of the UAV is constant, the shorter the flight length, the less time it takes to complete the flight mission.

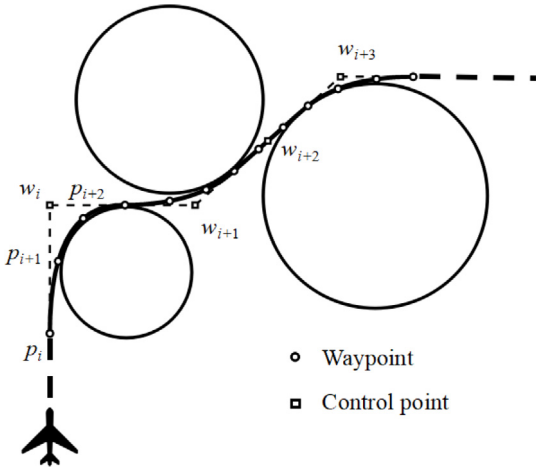


Fig. 5. Path smooth strategy.

The cost function J_L represents the length of the UAV path, which is calculated as follows:

$$J_L = \sum_{k=0}^{N} l_k \quad \text{with} \quad l_k = \sqrt{(x_{k+1}-x_k)^2 + (y_{k+1}-y_k)^2 + (z_{k+1}-z_k)^2} \tag{16}$$

(2) *Altitude cost*

The low altitude flight of the UAV facilitates the use of terrain cover to avoid enemy detection. Therefore, the cost function J_H integrates the flight altitude along the path to make the UAV tend to search the low-altitude routes, which is calculated as follows:

$$J_H = \int_{Path} H_p dl \quad \text{with} \quad H_p = \begin{cases} 0 & if\ z_k < 0 \\ z_p & otherwise \end{cases} \tag{17}$$

where z_p is the altitude of the UAV path at the waypoint p.

(3) *Threat constraint*

Three types of ground weapon units that threaten UAV flight are considered, i.e., radars, missiles, and anti-air guns. Their threat areas are represented in the form of cylinders. Corresponding detection probability and killing probability functions are calculated as follows.

The radar detection probability $P_{R,k}$ of the kth waypoint can be expressed by

$$P_{R,k} = \begin{cases} 0 & d > R_{Rmax} \\ \dfrac{R_{Rmax}^4}{d^4 + R_{Rmax}^4} & d \le R_{Rmax} \end{cases} \tag{18}$$

where R_{Rmax} is the radar's detection radius and d is the distance from the UAV to the radar detection center.

The killing probability $P_{M,k}$ of the kth waypoint of Ground-to-air missiles can be expressed by

$$P_{M,k} = \begin{cases} 0 & d > R_{Mmax} \\ \dfrac{R_{Mmax}^4}{d^4 + R_{Mmax}^4} & d \le R_{Mmax} \end{cases} \tag{19}$$

where R_{Mmax} is the maximum hit radius of Ground-to-air missile at a certain height and d is the distance between UAV and the missile center.

The killing probability $P_{G,k}$ of the kth waypoint of anti-aircraft gun can be expressed by

$$P_{G,k} = \begin{cases} 0 & d > R_{Gmax} \\ \dfrac{R_{Gmax}}{d + R_{Gmax}} & d \le R_{Gmax} \end{cases} \tag{20}$$

where R_{Gmax} denotes the maximum hit radius, and d is defined as the distance between UAV and the threat center.

The threat constraint function C_1 is the sum of all detection probabilities and killing probabilities, which are calculated as follows:

$$C_1 = \sum_{k=1}^{N} \left(P_{R,k} + P_{M,k} + P_{G,k}\right) \tag{21}$$

(4) *Forbidden flying area constraint*

The UAV should avoid entering some user-defined forbidden flying areas, such as harsh climate zones, unknown zones and so on. The forbidden flying area constraint function C_2 is calculated as follows:

$$C_2 = \sum_{k=1}^{N_z} \left(L_{in,k}\right) \tag{22}$$

where N_z is the number of forbidden flying areas, $L_{in,k}$ is the length of the UAV path that is inside the kth forbidden flying area.

(5) *Terrain constraint*

The minimum altitude of the UAV flight path above the ground should be greater than the predetermined safe flight height. The terrain constraint function C_3 is calculated as follows:

$$C_3 = \sum_{k=1}^{N} q_k \quad \text{with} \quad q_k = \begin{cases} 1 & if\ H_{safe} < z_k - H_{surf}\ (x_k, y_k) \\ 0 & otherwise \end{cases} \tag{23}$$

where H_{safe} is the minimal safe flight height, and $H_{surf}\ (x_k, y_k)$ is the height of the ground at (x_k, y_k).

(6) *Turning angle constraint*

The maximum turning angle ${\varphi_k}^{\max}$ is applied at each waypoint p_k to obtain the smoother routes, which depends on the maximum lateral overload n_{max} and the flight velocity V, given as

$$\varphi_k^{max} = \frac{n_{max} g}{V} \cdot \sqrt{(x_{k+1}-x_k)^2 + (y_{k+1}-y_k)^2} \tag{24}$$

where g is the gravitational acceleration. The turning angle constraint function C_4 is calculated as follows:

$$C_4 = \sum_{k=1}^{N} q_k \quad \text{with} \quad q_k = \begin{cases} 1 & if\ \varphi_k > \varphi_k^{max} \\ 0 & otherwise \end{cases} \tag{25}$$

where the turning angle φ_k is calculated as follows:

$$\phi_k = \pi - arccos \frac{l_{k-1,k}^2 + l_{k,k+1}^2 + l_{k+1,k-1}^2}{2 \times l_{k-1,k} \times l_{k,k+1}} \quad \text{with} \quad l_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{26}$$

(7) *Climbing & gliding slope constraint*

The UAV slope s_k at the waypoint p_k is constrained by the maximum climbing slope α_k and the minimum gliding slope β_k,

MCMOPSO-RL based path planning.

*/*Input for MCMOPSO-RL*/*
Set maximum number of iterations $t_{\max}$, the number of control points n and parameters $\lambda_{initial}$, λ_{final}, ω, γ, F.
*/*Initialization*/*
Set the generation number $t = 1$
Set the initial Q-table: $Q(s,a) = 0$
Initialize the location x_i and velocities v_i of particles randomly, $i = 1,2,...,NP$
*/*Evaluation*/*
Generate the smooth trajectory $Path=\{p_0, p_1, p_2, \ldots, p_N, p_{N+1}\}$
Evaluate flight path performance indicators by Eq. (31)
*/*Establish archives*/*
Non-dominated sorting and save non-dominated solutions to archives A
*/*Iteration computation*/*
While $t < t_{\max}$ **do**
  Calculate the learning rate λ by Eq. (9)
  Calculate mode conversion factor α by Eq. (15)
  For each particle i **do**
    Choose the best a for the current s from Q-table
    Switch action
      Case 1: Exploration update mode
        Update the velocity v_i and position x_i by Eq. (11) and Eq. (3)
      Case 2: Exploitation update mode
        Update the velocity v_i and position x_i by Eq. (12) and Eq. (3)
      Case 3: Hybrid update mode
        If $\alpha < rand$ **do**
          Update the position x_i by Eqs. (4)(5)(6)(14)
        Else if
          Update the position x_i by Eq. (13)
        End if
    End Switch
    Update the personal best position $x_{\text{pbest},i}$
    Calculate the reward r by Eq. (10)
  End for
  Update the Q-table
  Update the archives A
  $t = t + 1$
End while
*/*Output*/*
Output the best path for UAV.

Fig. 6. Procedure of the MCMOPSO-RL based path planner.

which both depend on the altitude. The constraint functions C_5 is calculated as follows:

$$C_5 = \sum_{k=1}^{N} q_k \quad \text{with} \quad q_k = \begin{cases} 1 & if\ s_k \geq \alpha_k \,||\, s_k \leq \beta_k \\ 0 & otherwise \end{cases} \tag{27}$$

where

$$\begin{aligned} \alpha_k &= -1.5377 \times 10^{-10}\, z_k^2 - 2.6997 \times 10^{-5}\, z_k + 0.4211 \\ \beta_k &= 2.5063 \times 10^{-9}\, z_k^2 - 6.3014 \times 10^{-6}\, z_k - 0.3257 \\ s_k &= \frac{z_{k+1} - z_k}{\sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2}} \end{aligned} \tag{28}$$

(8) *Collision constraint*

For the path planning of multiple UAVs, collisions between multiple UAVs must be avoided, which requires both spatial and

temporal constraints to be met. In other words, a safe distance must be maintained between UAVs at the same time. The coordinate of the waypoint p_i of the UAV to be calculated is (x_i, y_i, z_i). (x_j, y_j, z_j) is the coordinate of the waypoint p_j of another UAV. Therefore, the distance between the waypoints of the two UAVs is calculated as follows:

$$L_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \tag{29}$$

The collision constraint C_6 is calculated as follows:

$$C_6 = \sum_{i=1}^{N} \sum_{j=1}^{N} q_{ij}$$
with
$$q_{ij} = \begin{cases} 1 & if\ L_{ij} < L_{safe} \wedge |t_i - t_j| < t_{safe} \\ 0 & otherwise \end{cases} \tag{30}$$

where t_i and t_j are the time it takes for the UAVs fly to the waypoints p_i and p_j respectively, L_{safe} is the safe distance between UAVs, t_{safe} is the safe time to avoid collision.

Synthesizing the Eqs. (16)–(30), and taking constraints as an objective function, the mathematical model of the UAV path planning problem can be expressed as the following dual-objective optimization:

$$\min\ \boldsymbol{f} = (J, C)$$
with
$$J = J_{\mathrm{L}} + J_{\mathrm{H}}$$
$$C = \sum_{k=1}^{6} C_k \tag{31}$$

where J is the sum of flight cost, C is the sum of constraint cost. In order to generate a feasible UAV flight path, the value of the objective function C must be 0 at the end.

4.3. MCMOPSO-RL based path planner

The MCMOPSO-RL algorithm is applied to solve the single UAV path planning problem in three-dimensional complex environment. The specific implementation process is shown in Fig. 6.

4.4. Path planning for multiple UAVs

For solving the path planning problem of multiple UAVs, how to avoid collisions between them is the key. In this paper, we divide the constraint cost in two groups: (I) the ones related with each UAV (i.e., single-UAV cost) and (II) the cooperation cost (i.e., multi-UAVs cost). The individual cost includes threat constraint, forbidden flying area constraint, terrain constraint, turning angle constraint and climbing & gliding slope constraint. The collision constraints are divided into cooperation costs.

Our approach uses multiple populations (one per each UAV) to plan the path. Each population searches in parallel and exchanges some information with others during the evaluation phase. This information allows the algorithm to evaluate its own cooperation cost in consideration of other UAVs' best routes.

The calculation of the cooperation cost depends on the optimal path of other UAVs, while the optimality of the planned path needs to consider both individual costs and cooperation costs. It is impossible to directly send the optimal path of each UAV to other planners. Therefore, we divided the evaluation of individuals in the population into three steps:

Step 1. Each UAV only considers its own individual cost to determine the optimal solution and sends it to the others.

Step 2. Calculate the cooperation cost based on the received information.

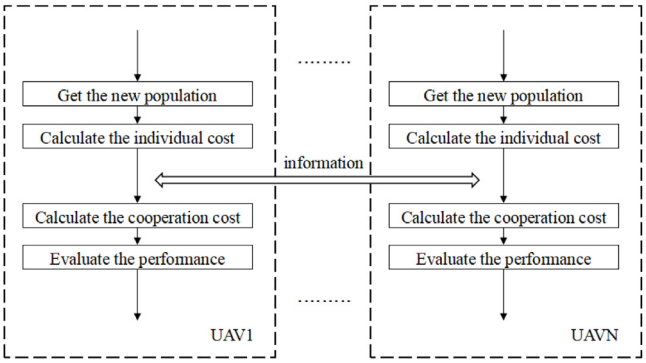


Fig. 7. The evaluation process of individuals.

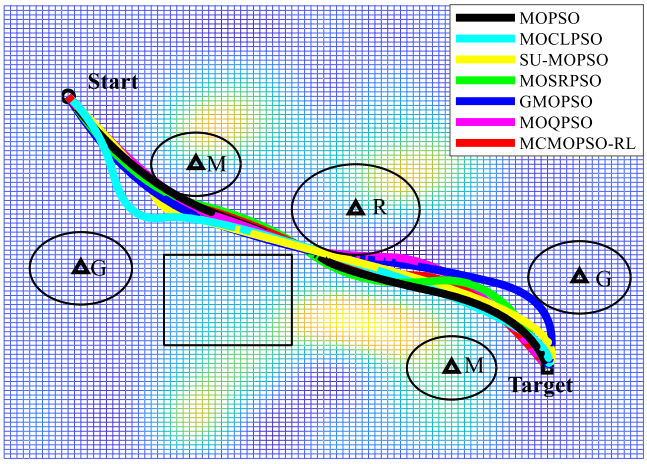


Fig. 8. Plane view of the best flight paths.

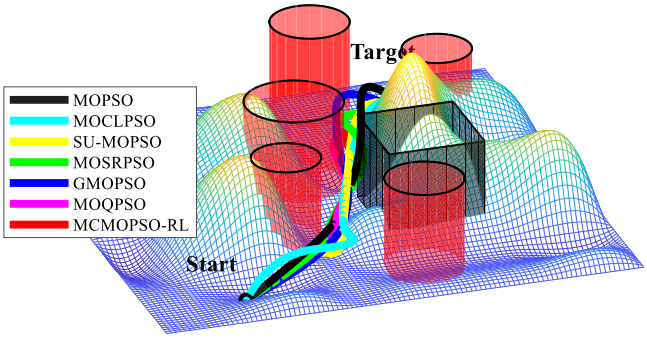


Fig. 9. The best flight paths in 3D environment.

Step 3. Comprehensively consider individual cost and cooperation cost to evaluate the performance of particles in the population.

The evaluation process of individuals is shown in Fig. 7.

5. Simulation results

In order to evaluate the performance of MCMOPSO-RL algorithm for UAV path planning, a series of experiments are conducted. In the complex three-dimensional virtual environment, we test the MCMOPSO-RL based path planning method. The experimental platform is a personal computer with Intel(R) Core(TM) i5, 2.30 GHz CPU, 8 GB memory, and Windows 10. The algorithm is implemented by Matlab-R2018a, and no commercial algorithm tools are used.

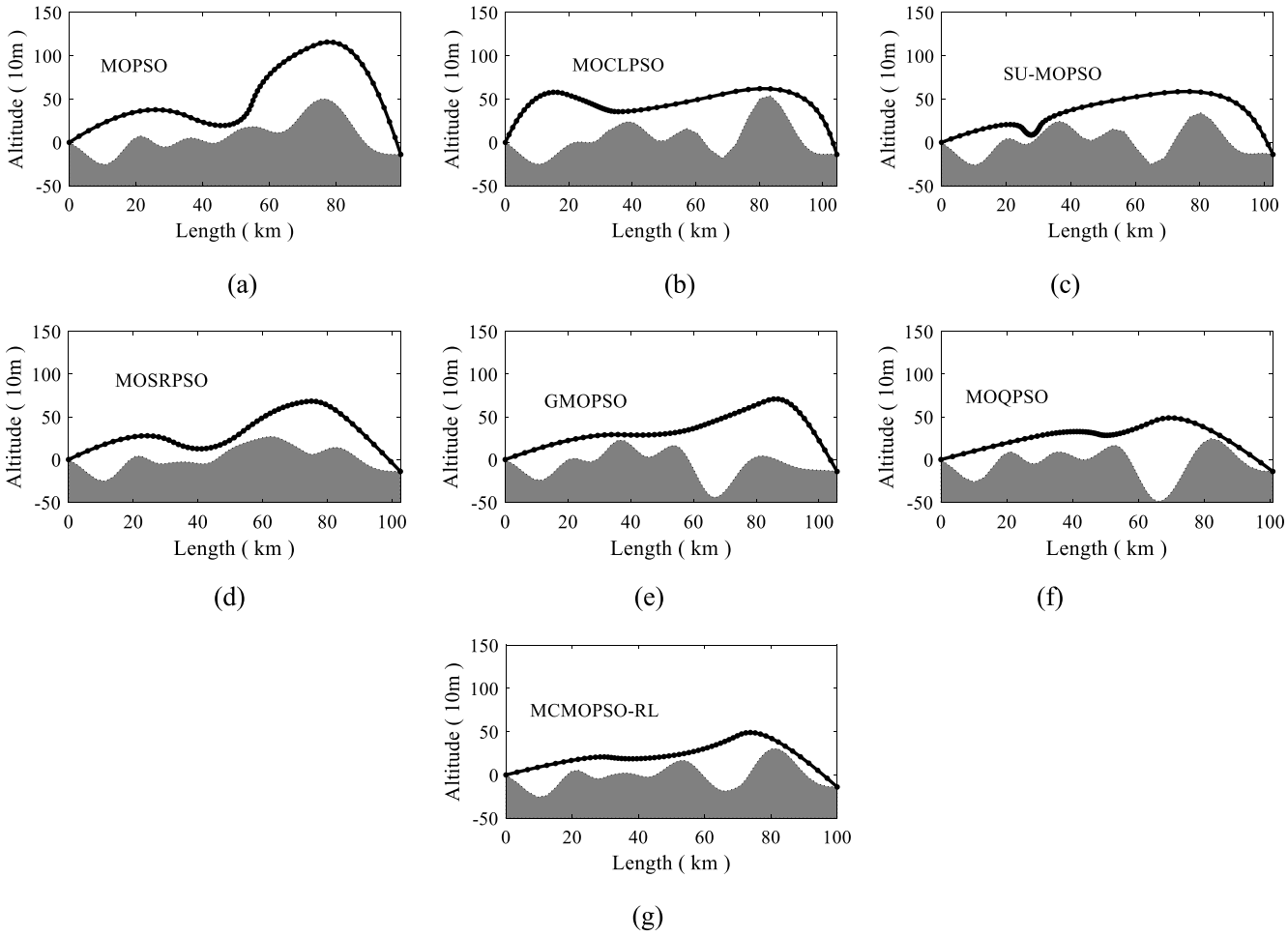


Fig. 10. The vertical profiles corresponding to the best paths in Fig. 8: (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

We compare proposed algorithm MCMOPSO-RL with other algorithms including MOPSO [39], MOCLPSO [40], SU-MOPSO [41], MOSRPSO [42], GMOPSO [43], and MOQPSO [44]. Each algorithm performs 50 times for path planning independently. The main parameters of MCMOPSO-RL are set as: maximum number of iterations $t_{\max} = 400$, population size $NP = 50$, initial and final learning rate $\lambda_{initial} = 0.9$, $\lambda_{final} = 0.1$, inertia weight $\omega = 0.4$, discount factor $\gamma = 0.5$, mutation factor $F = 0.9$. The relevant parameters of other algorithm are set according to the references.

5.1. Experiment for single UAV

For the path planning of single UAV, there is no need to consider collisions between UAVs. Therefore, we simulated a simple scenario similar to the valley, with $L_{\max} = 100$ km long and $W_{\max} = 100$ km wide. Threats and forbidden flying areas are set in the scene, and their range of influence is known. The parameters of UAV are set as follows: maximum flight altitude $H_{\max} = 4$ km, minimal safe flight height $H_{safe} = 50$ m, maximum lateral overload $n_{\max} = 5$, and flight speed $V = 200$ m/s.

Fig. 8 shows the plane view of the best flight paths obtained by the seven algorithms in 50 runs. Fig. 9 shows the best flight paths in the 3D environment. Inside the circle is the threat areas, and the rectangle represents the forbidden flying area. The labels R, M and G represent radar, missile and anti-aircraft gun respectively. Fig. 10 is vertical profiles corresponding to the best paths. Pareto fronts of the 50 runs are operated by Pareto sorting scheme. The finally obtained Pareto front is shown in Fig. 11.

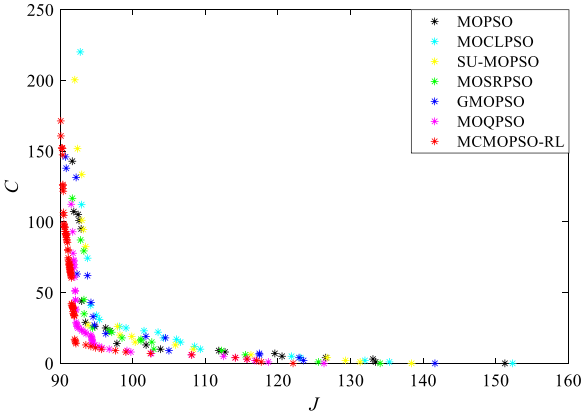


Fig. 11. Pareto front of all the algorithms for UAV path planning.

It can be seen that all the seven algorithms can avoid threats and obtain a safe path that meets performance constraints. However, compared with other algorithms, the best path obtained by MCMOPSO-RL algorithm is obviously smoother and better. From the perspective of the vertical profiles of the best paths, the path planned by the MCMOPSO-RL algorithm can maintain low-altitude flight. Intuitively, the proposed algorithm has good convergence.

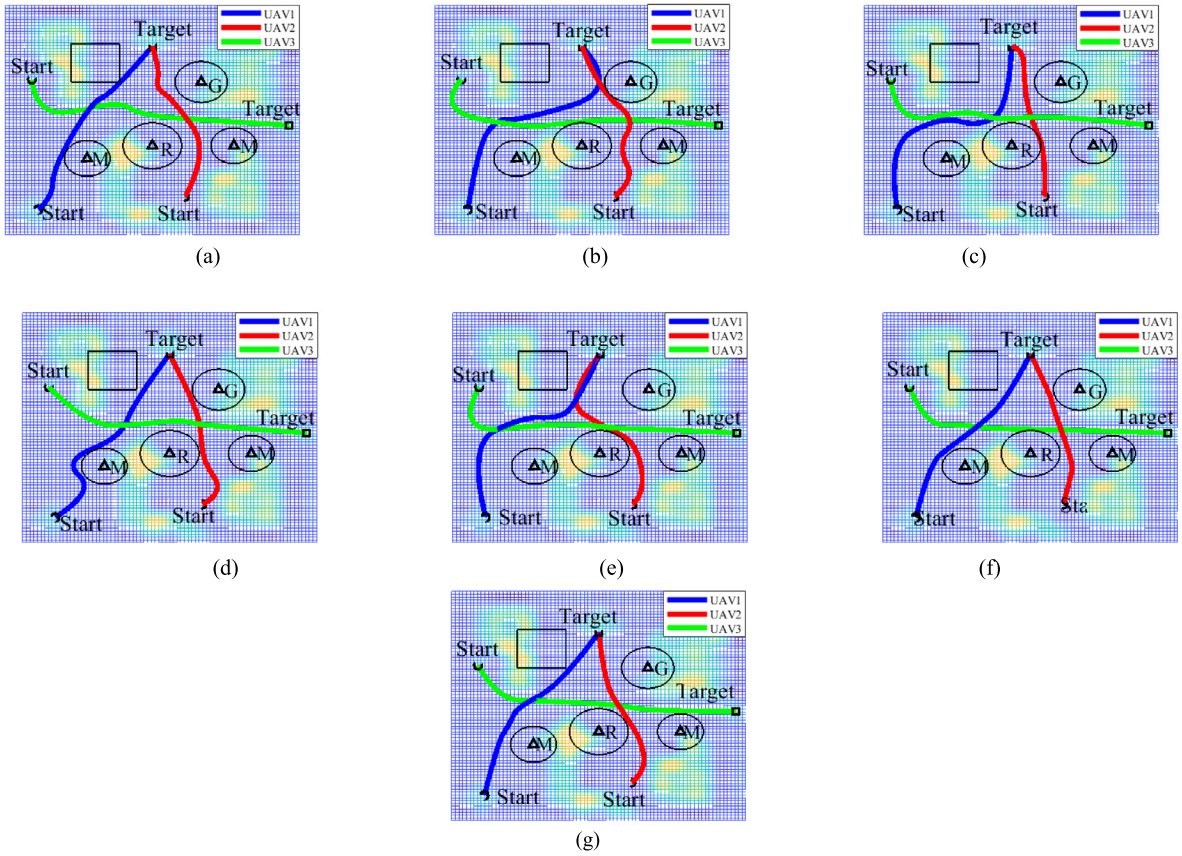


Fig. 12. Plane view of the best flight paths on Case 1. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

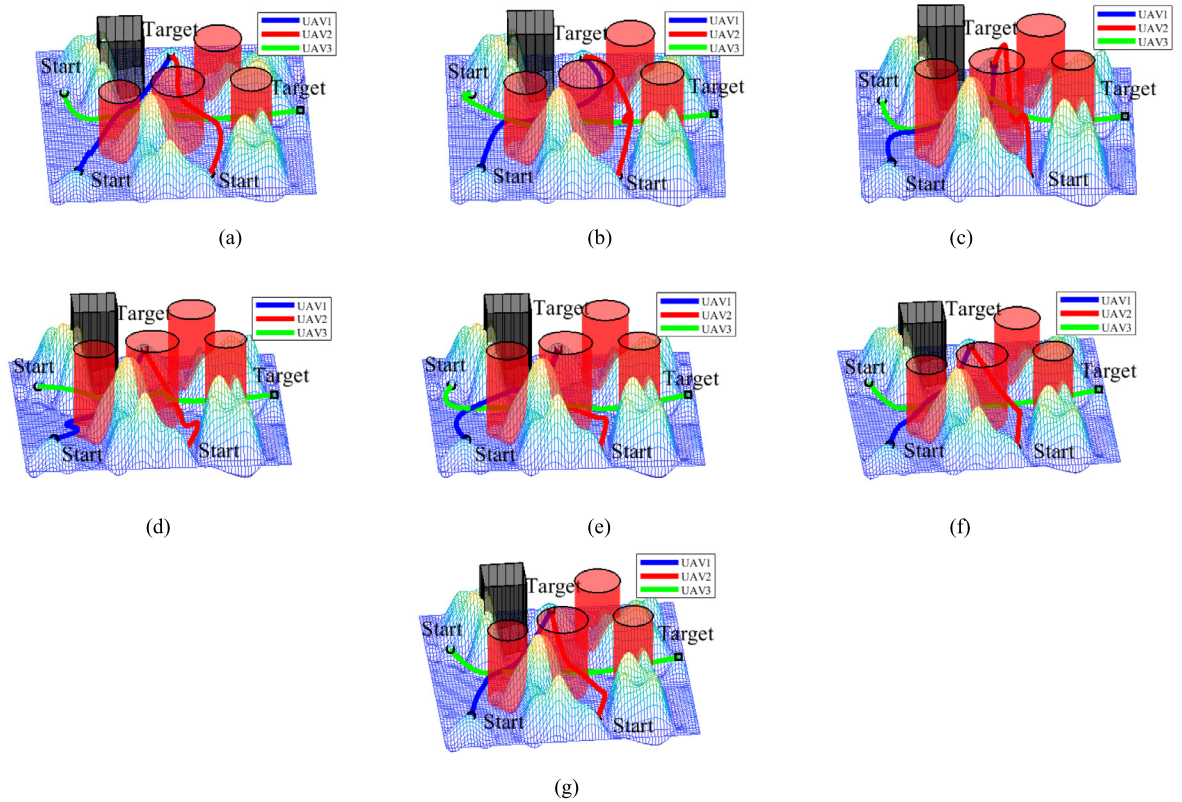


Fig. 13. The best flight paths in 3D environment on Case 1. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

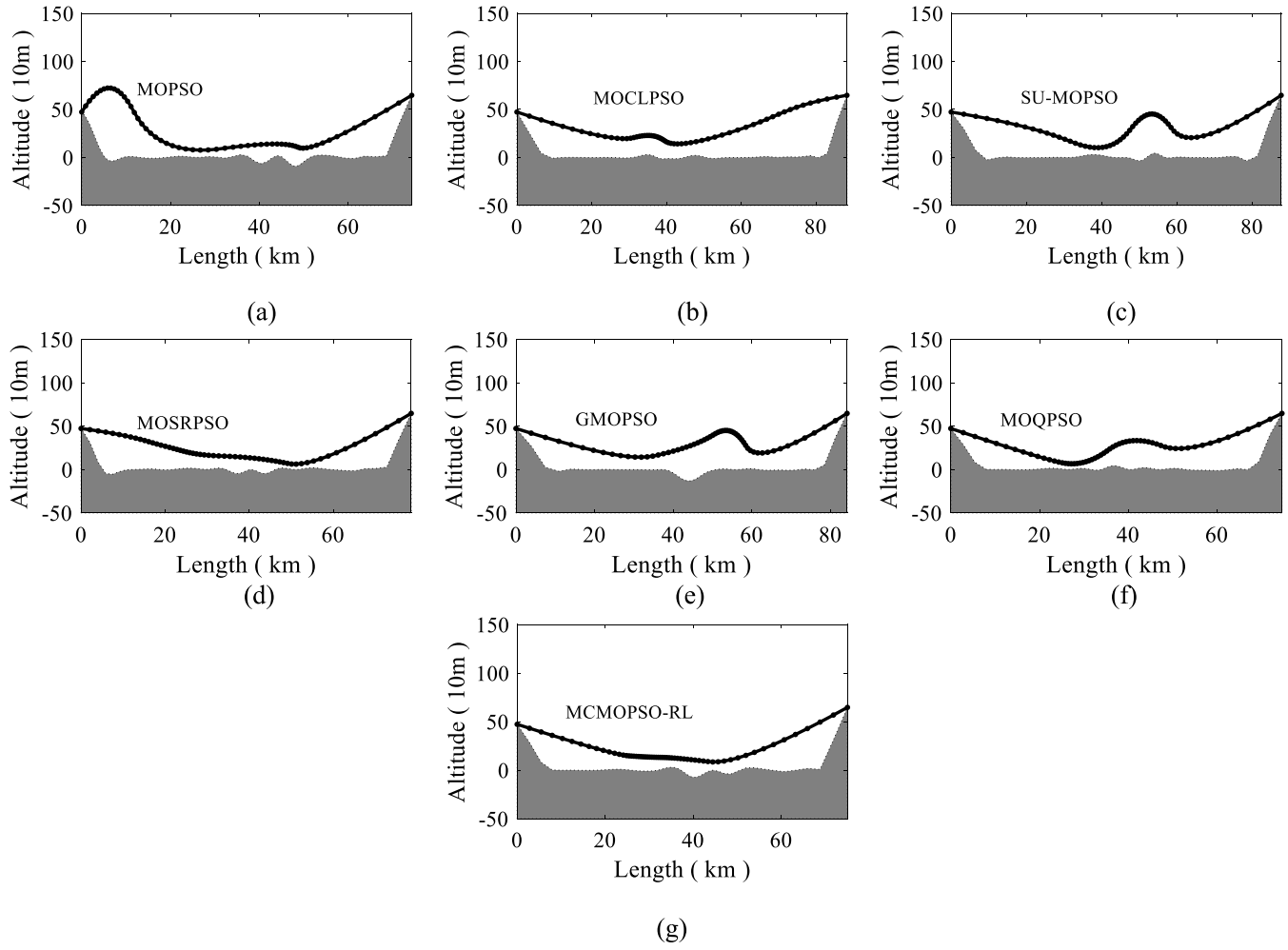


Fig. 14. The vertical profiles corresponding to the best paths of UAV1 on Case 1. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

Table 1
Performance of various algorithms for single UAV path planning.

Algorithm	Best	Median	Mean	Worst	Std.	FR (%)	AT (s)
MOPSO	151.27	168.14	179.38	267.20	31.67	64	76.24
MOCLPSO	152.34	194.25	207.21	405.91	45.79	56	75.96
SU-MOPSO	138.41	166.49	168.73	**202.52**	**15.83**	56	77.46
MOSRPSO	134.08	167.20	174.55	240.45	26.63	56	76.61
GMOPSO	141.63	192.49	214.83	425.42	63.67	82	77.40
MOQPSO	126.35	170.21	174.71	276.21	40.34	84	78.12
MCMOPSO-RL	**122.08**	**157.78**	**165.34**	245.02	31.13	**98**	**75.24**

Table 2
Performance comparison of various algorithms on Case 1.

Algorithm	Best	Median	Mean	Worst	Std.	FR (%)	AT (s)
MOPSO	279.48	289.70	294.50	383.45	**18.85**	66	182.93
MOCLPSO	358.02	438.48	446.86	554.09	93.31	6	185.69
SU-MOPSO	333.12	452.56	479.01	740.70	117.75	18	**170.41**
MOSRPSO	289.77	311.05	315.74	402.49	19.91	80	177.79
GMOPSO	308.61	349.06	371.40	710.15	76.99	94	178.99
MOQPSO	278.21	322.16	329.11	409.28	40.12	40	178.25
MCMOPSO-RL	**269.60**	**288.15**	**292.78**	**352.74**	19.88	**98**	180.43

As can be seen from Fig. 11, all seven algorithms can get a complete Pareto front. However, the Pareto front searched by MCMOPSO-RL is superior to the five comparison algorithms. The proposed algorithm shows the most satisfactory performance.

Table 1 lists the statistical results, recording the best, median, mean, worst and standard deviation of the flight cost value. The best results are marked in bold. FR represents the feasible rate of planning paths that satisfy the performance constraints from the results of 50 runs. AT denotes the average time for the algorithm to run once. Note that the statistics listed in columns 2–6 only calculate the successful operation of generating a safe path.

It can be seen that the overall performance of the MCMOPSO-RL to solve the UAV path planning problem is better than other comparison algorithms. The optimal flight cost of the path planned by MCMOPSO-RL is 122.08. It exceeds other algorithms obviously. Meanwhile, the FR of the proposed algorithm can reach 98%, 16% higher than the second highest, which is the reason for the poor values of worst and standard deviation. In addition, the median and average values of the paths planned by the proposed algorithm are the smallest. This proves that MCMOPSO-RL is effective and efficient, and it can successfully solve the path planning problem of single UAV in complex three-dimensional environment.

5.2. Experiment for multiple UAVs

For the path planning of multiple UAVs, the path planning between UAVs must be considered. To test the performance of MCMOPSO-RL, we designed two test cases. The settings of the scene and parameters of UAV are also the same as the single UAV test case.

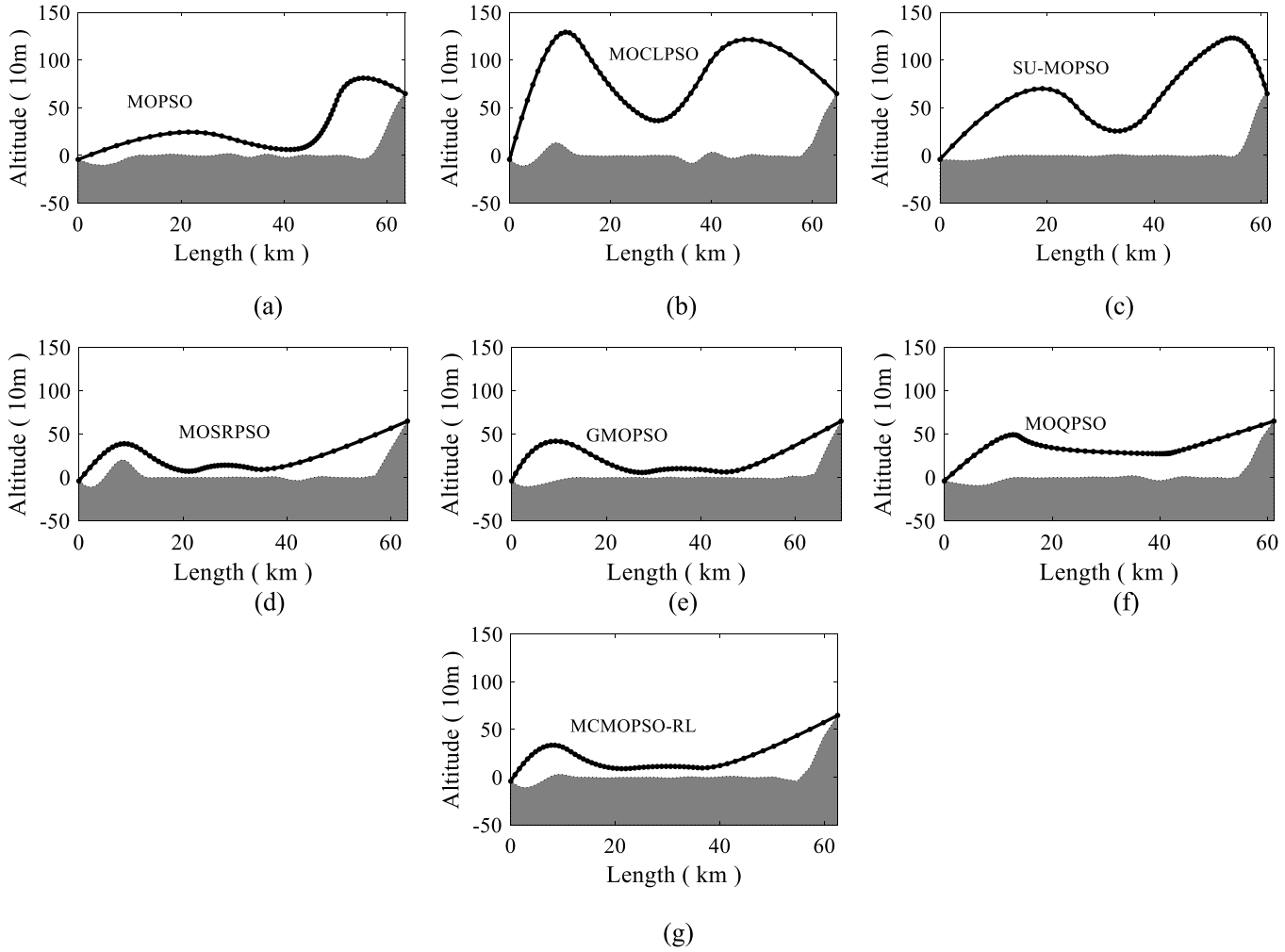


Fig. 15. The vertical profiles corresponding to the best paths of UAV2 on Case 1. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

Fig. 12 shows the plan view of the optimal flight path of Case 1 obtained by the seven algorithms in 50 runs, while the corresponding three-dimensional flight path is shown in Fig. 13. Figs. 14–16 are vertical profiles corresponding to the best paths of UAV1, UAV2 and UAV3 in Case 1 respectively. The Pareto fronts is shown in Fig. 17. Table 2 lists the statistical results of all algorithms of Case 1, including the best, median, mean, worst, and standard deviation of the total flight cost value. Table 3 lists the best, median, mean, worst, and standard deviation of the flight cost values of the three UAVs in Case 1, respectively.

It can be seen from the figures that the path planned by the MCMOPSO-RL algorithm is shorter and smoother, and can maintain low-altitude flight, which is superior to other algorithms. Perhaps the path of one of the UAVs planned by MCMOPSO-RL may not be significantly better than other algorithms, but in view of the planned paths of the three UAVs, the path planned by MCMOPSO-RL is optimal. The Pareto fronts of MCMOPSO-RL are closest to the real fronts, indicating that the MCMOPSO-RL algorithm is more convergent and can find the better solutions.

From the numerical results, the paths planned by MCMOPSO-RL algorithm for UAV2 is obviously superior to other algorithms. The best value of the paths planned by MCMOPSO-RL for UAV1 is 92.12, which is only 0.05 more than the minimum value planned by MOPSO. The median and mean values of the paths planned by MCMOPSO-RL are 99.77 and 97.34, which is the smallest compared with other comparison algorithms. In addition, the value of best cost and mean cost of the path planned for UAV3 are optimal. Furthermore, from the perspective of total flight cost, the

Table 3
Performance of various algorithms for each UAV on Case 1.

	Algorithm	Best	Median	Mean	Worst	Std.
UAV1	MOPSO	**92.07**	99.97	101.54	**120.69**	**6.25**
	MOCLPSO	117.58	135.40	159.52	225.56	57.89
	SU-MOPSO	107.83	124.42	137.66	266.80	49.46
	MOSRPSO	97.38	107.06	108.88	131.19	8.24
	GMOPSO	96.93	114.90	125.20	310.02	34.52
	MOQPSO	93.33	98.77	100.61	137.51	9.72
	MCMOPSO-RL	92.12	**97.34**	**99.77**	165.31	10.85
UAV2	MOPSO	76.75	85.22	87.57	177.40	16.47
	MOCLPSO	120.05	187.02	168.40	198.12	42.24
	SU-MOPSO	99.15	211.00	221.81	353.26	82.44
	MOSRPSO	75.91	89.49	93.70	186.98	17.67
	GMOPSO	85.42	99.91	106.87	341.66	36.59
	MOQPSO	77.51	98.91	106.64	195.02	29.69
	MCMOPSO-RL	**75.07**	**82.68**	**84.86**	**123.01**	**7.98**
UAV3	MOPSO	99.51	103.33	**105.40**	130.42	5.84
	MOCLPSO	116.06	120.39	118.95	**120.41**	**2.50**
	SU-MOPSO	110.02	118.29	119.53	132.51	7.82
	MOSRPSO	104.02	111.94	113.17	128.68	5.86
	GMOPSO	105.45	126.07	139.33	280.03	38.63
	MOQPSO	98.83	108.31	121.86	224.85	36.04
	MCMOPSO-RL	**98.12**	**101.82**	108.16	181.73	16.71

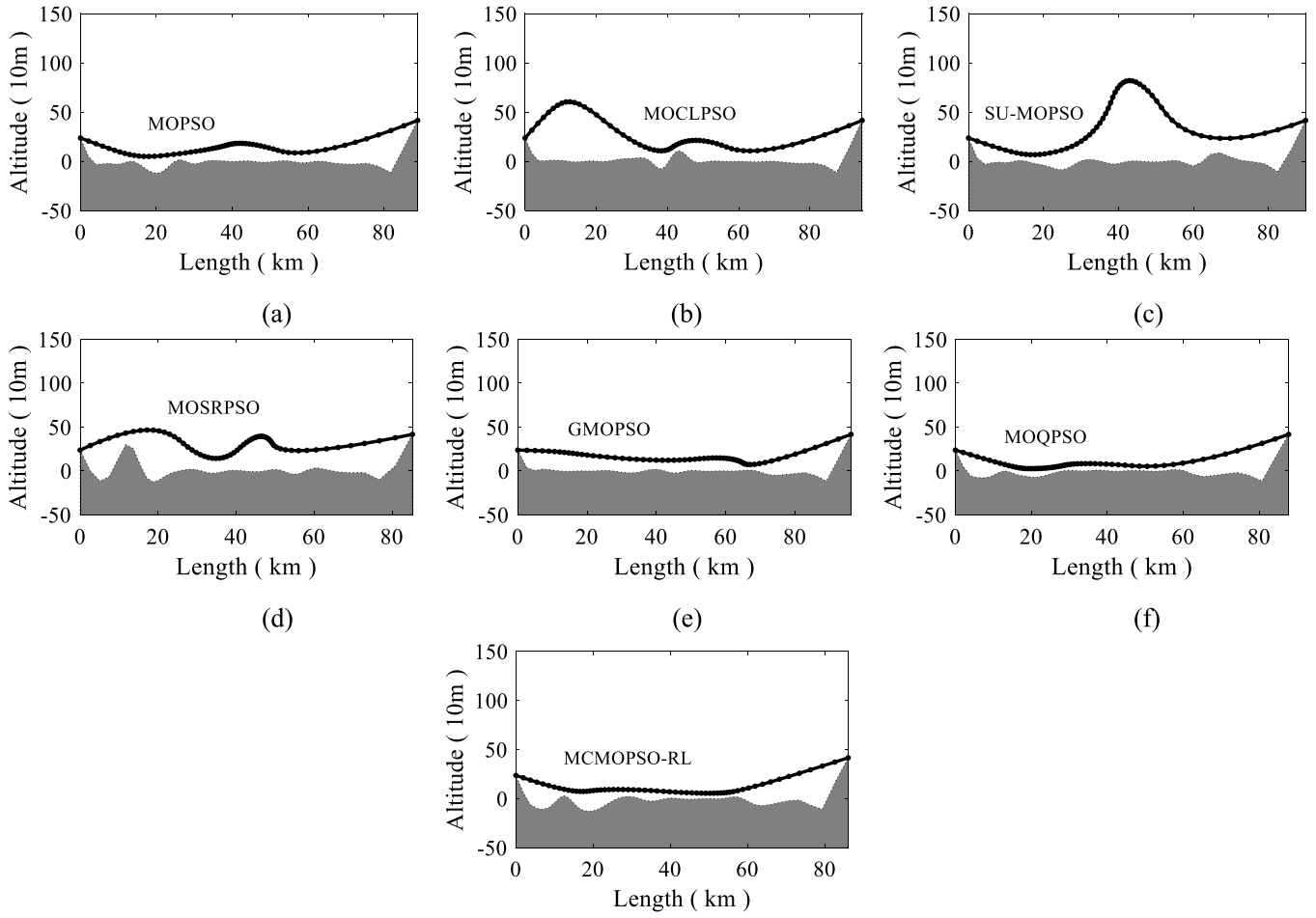


Fig. 16. The vertical profiles corresponding to the best paths of UAV3 on Case 1. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

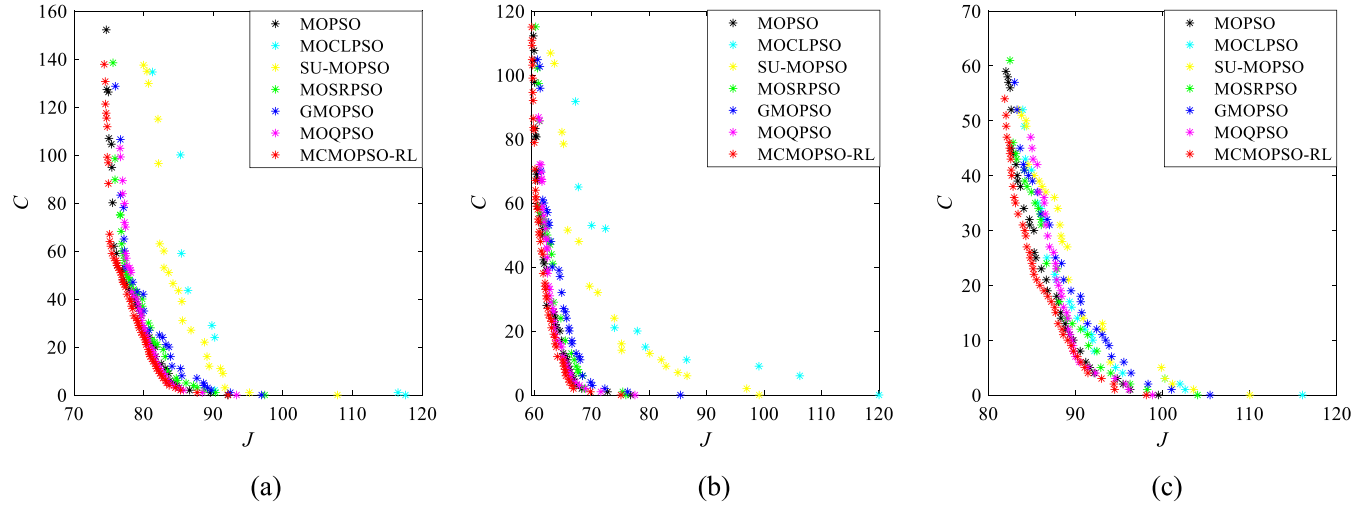


Fig. 17. Pareto front of all the algorithms for UAV path planning. (a) UAV1. (b) UAV2. (c) UAV3.

best, median, mean and worst values of the paths planned by the MCMOPSO-RL algorithm are the smallest. The value of standard deviation is 19.88, second only to MOPSO. More importantly, the FR of path planning using MCMOPSO-RL algorithm is the highest, reaching 98%.

Fig. 18 shows the plan view of the optimal flight path of Case 2, and Fig. 19 shows the corresponding three-dimensional flight path. Figs. 20–22 are the vertical profiles corresponding to the optimal paths of different UAVs in Case 2. The Pareto fronts are shown in Fig. 23. Table 4 lists the statistical results of all

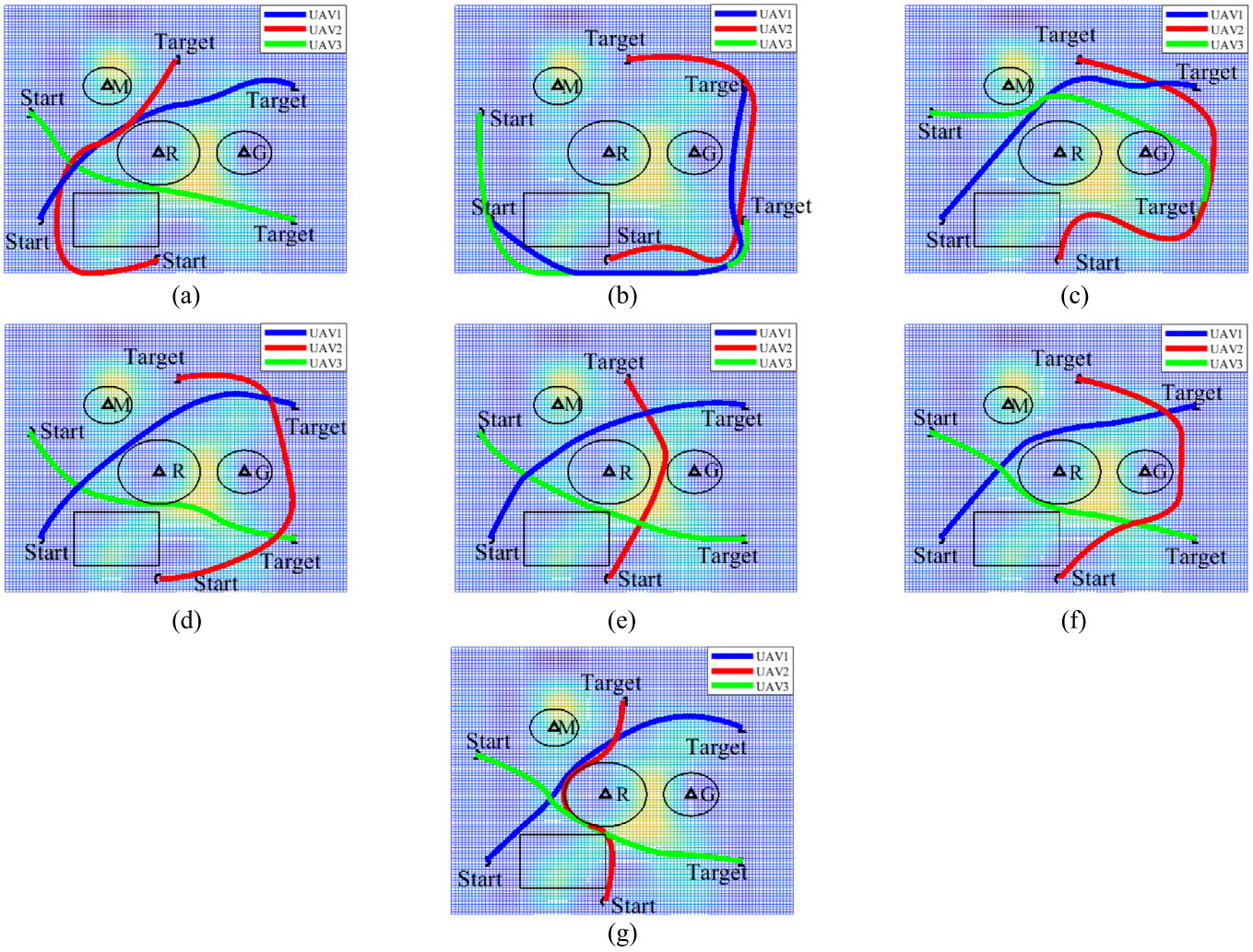


Fig. 18. Plane view of the best flight paths on Case 2. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

Table 4
Performance comparison of various algorithms on Case 2.

Algorithm	Best	Median	Mean	Worst	Std.	FR (%)	AT (s)
MOPSO	471.42	531.62	534.34	653.19	43.52	42	203.77
MOCLPSO	608.32	701.09	708.25	878.24	86.88	14	205.79
SU-MOPSO	606.94	662.23	675.48	885.41	83.35	18	213.01
MOSRPSO	508.52	549.39	560.56	677.47	37.06	78	**202.47**
GMOPSO	477.45	514.31	527.01	690.27	39.35	**100**	242.37
MOQPSO	498.47	582.88	577.91	700.19	54.91	**34**	204.98
MCMOPSO-RL	**459.38**	**499.65**	**508.57**	**634.33**	**33.55**	**100**	203.99

Table 5
Performance of various algorithms for each UAV on Case 2.

	Algorithm	Best	Median	Mean	Worst	Std.
UAV1	MOPSO	139.90	154.62	163.48	281.95	31.41
	MOCLPSO	170.44	211.68	209.58	232.00	19.82
	SU-MOPSO	175.73	207.49	211.25	276.83	29.25
	MOSRPSO	145.00	162.46	167.36	231.49	19.81
	GMOPSO	134.16	152.90	159.40	226.50	20.97
	MOQPSO	142.47	155.56	166.85	245.46	28.91
	MCMOPSO-RL	**132.88**	**148.72**	**149.04**	**185.16**	**10.70**
UAV2	MOPSO	155.94	175.05	181.93	260.31	**22.36**
	MOCLPSO	207.49	243.10	276.34	435.32	77.58
	SU-MOPSO	219.66	254.14	269.44	398.75	54.34
	MOSRPSO	160.72	186.15	191.56	312.28	25.91
	GMOPSO	**144.01**	175.14	186.06	369.29	35.68
	MOQPSO	170.77	206.05	206.04	**249.98**	27.28
	MCMOPSO-RL	150.99	**172.23**	**176.90**	306.42	24.39
UAV3	MOPSO	172.54	184.36	188.93	**218.83**	**13.81**
	MOCLPSO	189.16	214.34	222.33	282.39	32.00
	SU-MOPSO	166.47	191.66	194.80	220.24	15.99
	MOSRPSO	173.52	193.20	201.65	255.52	20.79
	GMOPSO	158.08	**176.21**	**181.54**	240.55	16.92
	MOQPSO	**155.41**	190.37	205.01	279.64	34.86
	MCMOPSO-RL	159.74	177.18	182.64	254.85	20.16

algorithms of Case 2. Table 5 lists the statistical results of the flight cost for different UAVs in Case 2.

It can be seen from the figures that the paths planned by MCMOPSO-RL does not take detours, is smoother, and is superior to the path planned by other algorithms. From the perspective of the Pareto fronts, MCMOPSO-RL also shows satisfactory performance. Although the quality of the Pareto fronts generated by MOQPSO is similar to that of the MCMOPSO-RL algorithm, the planned path is far inferior to MCMOPSO-RL.

According to the numerical results, the path planned by MCMOPSO-RL is better than other algorithms as a whole. Meanwhile, the proposed algorithm can successfully plan the flight

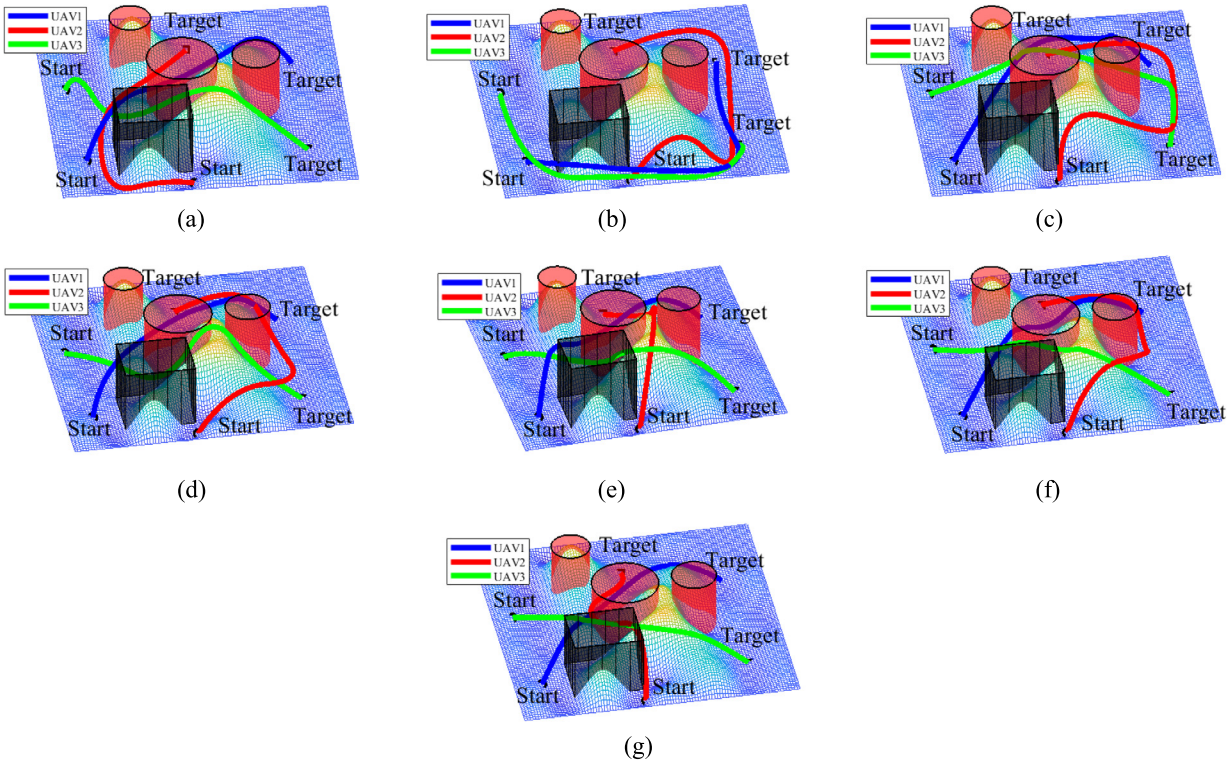


Fig. 19. The best flight paths in 3D environment on Case 2. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

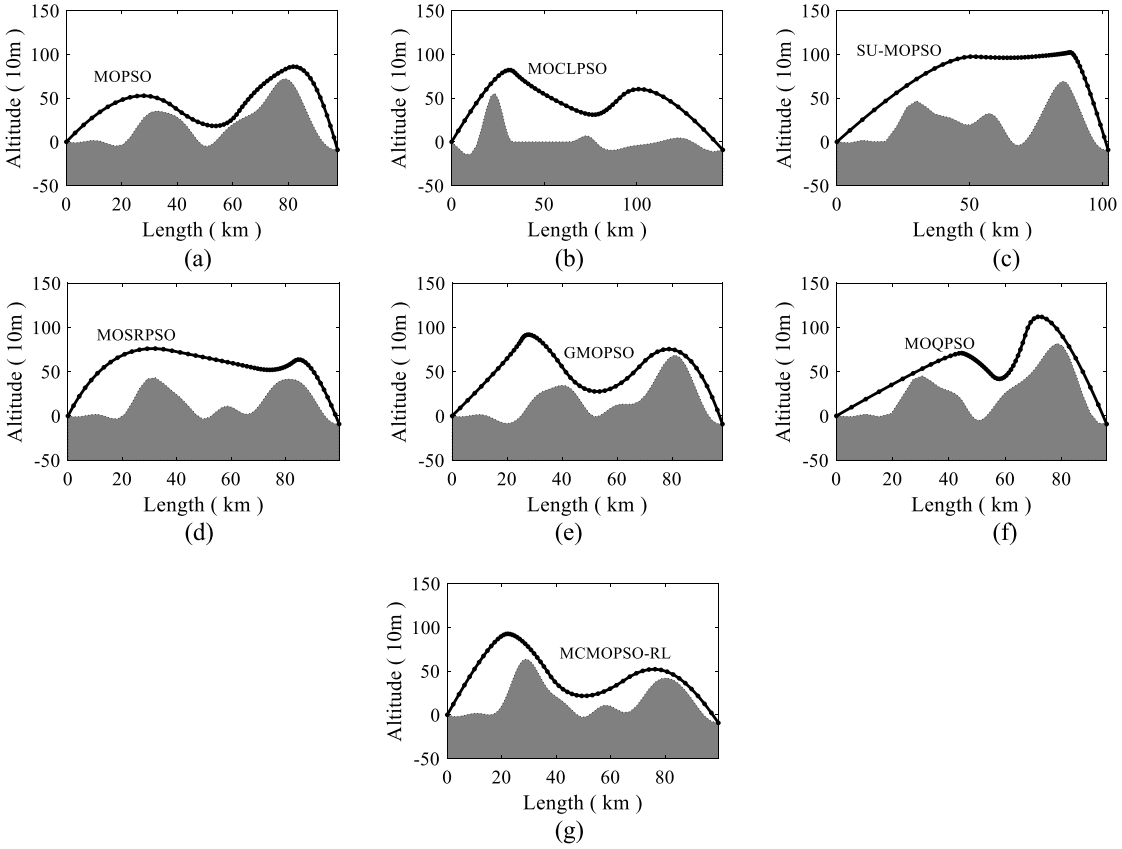


Fig. 20. The vertical profiles corresponding to the best paths of UAV1 on Case 2. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

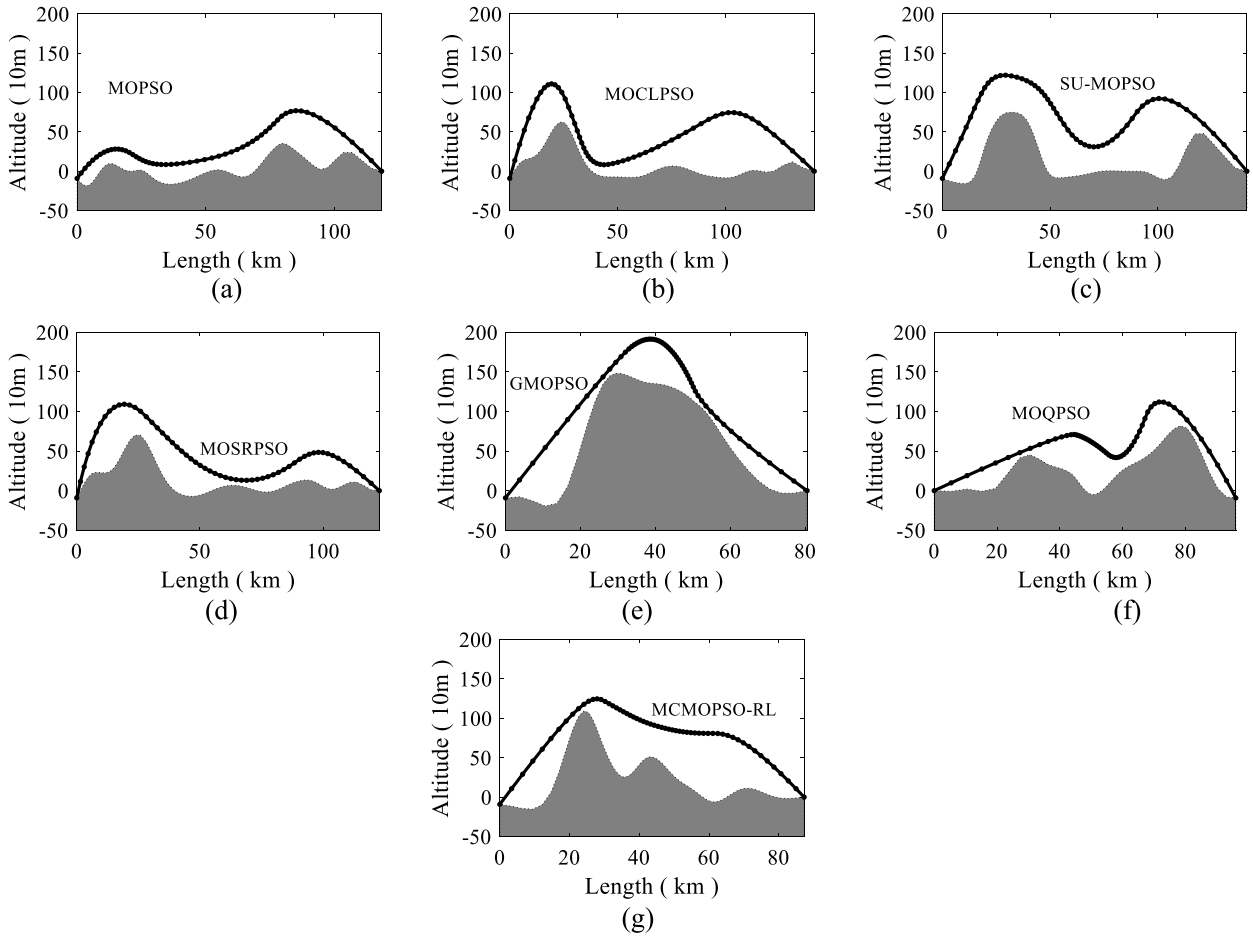


Fig. 21. The vertical profiles corresponding to the best paths of UAV2 in Case 2. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

paths of the UAVs every time. In addition, MCMOPSO-RL is completely superior to other algorithms for the planning of UAV1. It can be seen from Fig. 18(g) that UAV3 and UAV2 have a part of almost overlapping paths, which requires comprehensive consideration of the flight costs of the two UAVs to avoid collisions. For the planning of UAV3, MCMOPSO-RL is slightly inferior to the optimal GMOPSO, but its planning for UAV2 is significantly better. The total cost of the path of UAV2 and UAV3 planned by MCMOPSO-RL is smaller. Therefore, although the performance of UAV3 is not the best, from the perspective of the overall planning of the three UAVs, the planning of MCMOPSO-RL is undoubtedly more reasonable.

The experimental results show that the MCMOPSO-RL algorithm can be successfully applied to solve the path planning problem of single and multiple UAVs. In two test cases, the proposed algorithm showed strong search ability and robustness. The algorithm can plan safe, feasible, and smooth flight path for the UAVs. This proves that the proposed algorithm is effective, efficient and robust. Compared with other algorithms, MCMOPSO-RL has better performance in solving UAV path planning problems.

6. Conclusion

The path planning problem of multiple UAVs in three-dimensional complex environment is usually regarded as a constrained optimization problem with multiple constraints. In order to effectively solve this problem, multi-objective optimization technique is adopted to handle the constraints in this paper. By treating all constraints as an objective function, the dual-objective optimization model is established. Then, we propose a multi-objective particle swarm optimization algorithm with multi-mode collaboration based on reinforcement learning to solve this optimization problem.

Aiming at the problem of insufficient individual characteristics in some existing multi-objective particle swarm algorithms, we use a reinforcement learning strategy so that each particle in the population can choose an appropriate update mode according to its own state and performance. Meanwhile, three update modes are developed for each particle. The exploration and exploitation modes respectively increase the diversity of the population and accelerated the convergence rate. As a supplement, the hybrid mode is a routine search operation, which improves the robustness of the algorithm.

The proposed algorithm is tested in scenarios for single and multiple UAV path planning. Compared with some other existing improved algorithms, the proposed algorithm generates a higher quality Pareto front. Meanwhile, the path planned by the proposed algorithm can make the UAVs reach the targets faster and more safely without violating any performance constraints, which proves its effectiveness and efficiency. Experiments show that the proposed algorithm has better performance in solving UAV path planning problem. In the future, we will further improve the proposed algorithm in terms of the initialization of the population, the adjustment of the parameters in the algorithm, and so on. In

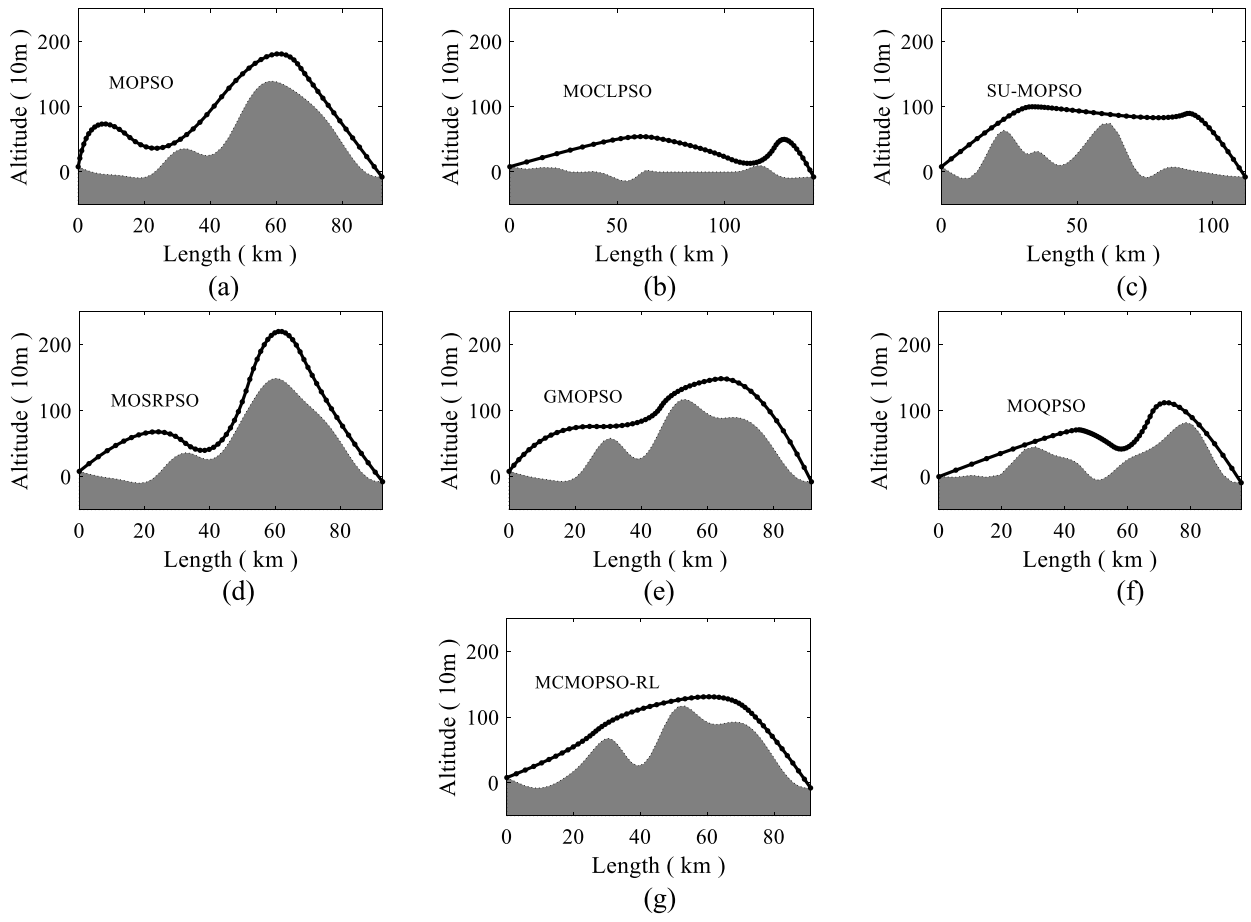


Fig. 22. The vertical profiles corresponding to the best paths of UAV3 in Case 2. (a) MOPSO. (b) MOCLPSO. (c) SU-MOPSO. (d) MOSRPSO. (e) GMOPSO. (f) MOQPSO. (g) MCMOPSO-RL.

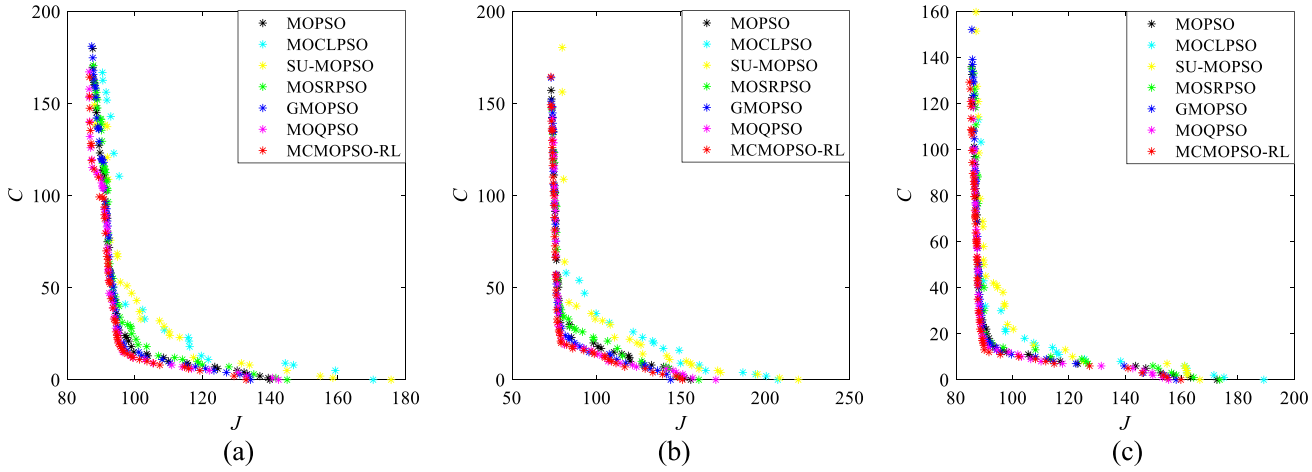


Fig. 23. Pareto front of all the algorithms for UAV path planning. (a) UAV1. (b) UAV2. (c) UAV3.

addition, the path planning of UAVs in a dynamic environment with moving obstacles will also be researched in our following work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by National Natural Science Foundation of China (No. 61703012, 51975011) and Beijing Natural Science Foundation, China (No. 4182010).

References

[1] Y. Zhao, Z. Zheng, Y. Liu, Survey on computational-intelligence-based UAV path planning, Knowl.-Based Syst. 158 (2018) 54–64.

[2] H. Yu, K. Meier, M. Argyle, R.W. Beard, Cooperative path planning for target tracking in urban environments using unmanned air and ground vehicles, IEEE/ASME Trans. Mechatronics 20 (2) (2015) 541–552.
[3] P. Razi, J.T.S. Sumantyo, D. Perissin, H. Kuze, M.Y. Chua, G.F. Panggabean, 3D land mapping and land deformation monitoring using persistent scatterer interferometry (PSI) ALOS PALSAR: validated by geodetic GPS and UAV, IEEE Access 6 (2018) 12395–12404.
[4] E.T. Alotaibi, S.S. Alqefari, A. Koubaa, LSAR: Multi-UAV collaboration for search and rescue missions, IEEE Access 7 (2019) 55817–55832.
[5] C. Qu, W. Gai, J. Zhang, M. Zhong, A novel hybrid grey wolf optimizer algorithm for unmanned aerial vehicle (UAV) path planning, Knowl.-Based Syst. 194 (2020) 105530.
[6] Y. Du, X. Zhang, Z. Nie, A real-time collision avoidance strategy in dynamic airspace based on dynamic artificial potential field algorithm, IEEE Access 7 (2019) 169469–169479.
[7] Y. Lin, S. Saripalli, Sampling-based path planning for UAV collision avoidance, IEEE Trans. Intell. Transp. Syst. 18 (11) (2017) 3179–3192.
[8] Y. Liu, X.J. Zhang, X.M. Guan, D. Delahaye, Adaptive sensitivity decision based path planning algorithm for unmanned aerial vehicle with improved particle swarm optimization, Aerosp. Sci. Technol. 58 (2016) 92–102.
[9] Z.Y. Jia, J.Q. Yu, X.L. Ai, X. Xu, D. Yang, Cooperative multiple task assignment problem with stochastic velocities and time windows for heterogeneous unmanned aerial vehicles using a genetic algorithm, Aerosp. Sci. Technol. 76 (2018) 112–125.
[10] Q. Zhang, S.W. Xiong, Routing optimization of emergency grain distribution vehicles using the immune ant colony optimization algorithm, Appl. Soft Comput. 71 (2018) 917–925.
[11] D.P. Kong, T.Q. Chang, W.J. Dai, Q.D. Wang, H.Z. Sun, An improved artificial bee colony algorithm based on elite group guidance and combined breadth-depth search strategy, Inform. Sci. 442–443 (2018) 54–71.
[12] X.Y. Zhang, X.Y. Lu, S.M. Jia, X.Z. Li, A novel phase angle-encoded fruit fly optimization algorithm with mutation adaptation mechanism applied to UAV path planning, Appl. Soft Comput. 70 (2018) 371–388.
[13] X.Y. Zhang, H.B. Duan, An improved constrained differential evolution algorithm for unmanned aerial vehicle global route planning, Appl. Soft Comput. 26 (2015) 270–284.
[14] X. Yu, C. Li, J. Zhou, A constrained differential evolution algorithm to solve UAV path planning in disaster scenarios, Knowl.-Based Syst. 204 (2020) 106209.
[15] K. Li, F.W. Ge, Y. Han, Y.A. Wang, W.S. Xu, Path planning of multiple UAVs with online changing tasks by an ORPFOA algorithm, Eng. Appl. Artif. Intell. 94 (2020) in press.
[16] Y. Liu, X.J. Zhang, Y. Zhang, X.M. Guan, Collision free 4D path planning for multiple UAVs based on spatial refined voting mechanism and PSO approach, Chin. J. Aeronaut. 32 (6) (2019) 1504–1519.
[17] M. Radmanesh, M. Kumar, M. Sarim, Grey wolf optimization based sense and avoid algorithm in a Bayesian framework for multiple UAV path planning in an uncertain environment, Aerosp. Sci. Technol. 77 (2018) 168–179.
[18] H. Zhou, X. Zeng, Y. Hong, Adaptive exact penalty design for constrained distributed optimization, IEEE Trans. Automat. Control 64 (11) (2019) 4661–4667.
[19] A. Saha, R. Datta, K. Deb, Hybrid gradient projection based genetic algorithms for constrained optimization, in: Proceedings of 2010 IEEE Congress on Evolutionary Computation, Barcelona, 2010, pp. 1–8.
[20] Y. Wang, Z. Cai, Combining multiobjective optimization with differential evolution to solve constrained optimization problems, IEEE Trans. Evol. Comput. 16 (1) (2012) 117–134.
[21] M. Zhao, R. Liu, W. Li, H. Liu, Multi-objective optimization based differential evolution constrained optimization algorithm, in: Proceedings of 2010 Second WRI Global Congress on Intelligent Systems, Wuhan, 2010, pp. 320–326.
[22] R. Chai, A. Savvaris, A. Tsourdos, Y. Xia, S. Chai, Solving multiobjective constrained trajectory optimization problem by an extended evolutionary algorithm, IEEE Trans. Cybern. 50 (4) 1630–1643.
[23] R. Chai, A. Savvaris, A. Tsourdos, S. Chai, Multi-objective trajectory optimization of Space Manoeuvre Vehicle using adaptive differential evolution and modified game theory, Acta Astronaut. 136 (2017) 273–280.
[24] M.A. Mosa, A novel hybrid particle swarm optimization and gravitational search algorithm for multi-objective optimization of text mining, Appl. Soft Comput. 90 (2020) 106189.
[25] T. Liu, L. Jiao, W. Ma, J. Ma, R. Shang, A new quantum-behave d particle swarm optimization based on cultural evolution mechanism for multiobjective problems, Knowl.-Based Syst. 101 (2016) 90–99.
[26] C.A.C. Coello, M.S. Lechuga, MOPSO: a proposal for multiple objective particle swarm optimization, Evol. Comput. (2002) 1051–1056.
[27] T.T. Mac, C. Copot, D.T. Tran, R. De Keyser, A hierarchical global path planning approach for mobile robots based on multi-objective particle swarm optimization, Appl. Soft Comput. 59 (2017) 68–76.
[28] S.Z. Zhao, P.N. Suganthan, Two-lbests based multi-objective particle swarm optimizer, Eng. Optim. 43 (1) (2011) 1–17.
[29] K.M. Ang, W.H. Lim, N.A.M. Isa, S.S. Tiang, C.H. Wong, A constrained multi-swarm particle swarm optimization without velocity for constrained optimization problems, Expert Syst. Appl. 140 (2020) 112882.
[30] Z.K. Feng, W.J. Niu, C.T. Cheng, Multi-objective quantum-behaved particle swarm optimization for economic environmental hydrothermal energy system scheduling, Energy 131 (2017) 165–178.
[31] H. Qie, D. Shi, T. Shen, X. Xu, Y. Li, L. Wang, Joint optimization of multi-UAV target assignment and path planning based on multi-agent reinforcement learning, IEEE Access 7 (2019) 146264–146272.
[32] B. Li, Y. Wu, Path planning for UAV ground target tracking via deep reinforcement learning, IEEE Access 8 (2020) 29064–29074.
[33] X. Zhou, P. Wu, H. Zhang, W. Guo, Y. Liu, Learn to navigate: cooperative path planning for unmanned surface vehicles using deep reinforcement learning, IEEE Access 7 (2019) 165262–165278.
[34] C.Z. Qu, W.D. Gai, M.Y. Zhong, J. Zhang, A novel reinforcement learning based grey wolf optimizer algorithm for unmanned aerial vehicles (UAVs) path planning, Appl. Soft Comput. 89 (2020) 106099.
[35] R. Chai, A. Tsourdos, A. Savvaris, Y. Xia, S. Chai, Real-time reentry trajectory planning of hypersonic vehicles: A two-step strategy incorporating fuzzy multiobjective transcription and deep neural network, IEEE Trans. Ind. Electron. 67 (8) (2020) 6904–6915.
[36] R. Chai, A. Tsourdos, A. Savvaris, S. Chai, Y. Xia, C.L.P. Chen, Six-DOF spacecraft optimal trajectory planning and real-time attitude control: A deep neural network-based approach, IEEE Trans. Neural Netw. Learn. Syst..
[37] J. Sun, W.B. Xu, B. Feng, Adaptive parameter control for quantum-behaved particle swarm optimization on individual level, in: Proceedings of 2005 IEEE International Conference on Systems, Man and Cybernetics, Waikoloa, HI, Vol. 4, 2005, pp. 3049–3054.
[38] L.S. Coelho, Novel Gaussian quantum-behaved particle swarm optimiser applied to electromagnetic design, IET Sci. Meas. Technol. 1 (5) (2007) 290–294.
[39] M.E.B. Aguilar, D.V. Coury, R. Reginatto, R.M. Monaro, Multi-objective PSO applied to PI control of DFIG wind turbine under electrical fault conditions, Electr. Power Syst. Res. 180 (2020) 106081.
[40] V.L. Huang, P.N. Suganthan, J.J. Liang, Comprehensive learning particle swarm optimizer for solving multiobjective optimization problems: Research articles, Int. J. Intell. Syst. 21 (2) (2006) 209–226.
[41] J.G. Wang, W.J. Liu, W.X. Zhang, B. Yang, Multi-objective particle swarm optimization algorithm based on self-update strategy, in: Proceedings of 2012 International Conference on Industrial Control and Electronics Engineering, Xi'an, 2012, pp. 171–174.
[42] M.R. Tanweer, A. Al-Dujaili, S. Suresh, Multi-objective self regulating particle swarm optimization algorithm for BMOBench platform, in: 2016 IEEE Symposium Series on Computational Intelligence (SSCI), Athens, 2016, pp. 1–6.
[43] L. dos Santos Coelho, H.V.H. Ayala, P. Alotto, A multiobjective Gaussian particle swarm approach applied to electromagnetic optimization, IEEE Trans. Magn. 46 (8) (2010) 3289–3292.
[44] J. Wang, G. Jia, J. Lin, Z. Hou, Cooperative mission planning for heterogeneous UAVs with the improved multi-objective quantum-behaved particle swarm optimization algorithm, in: 2019 Chinese Control and Decision Conference (CCDC), Nanchang, China, 2019, pp. 3740–3745.